



Desenvolvimento Web

Profª Sabrina B. Moreira



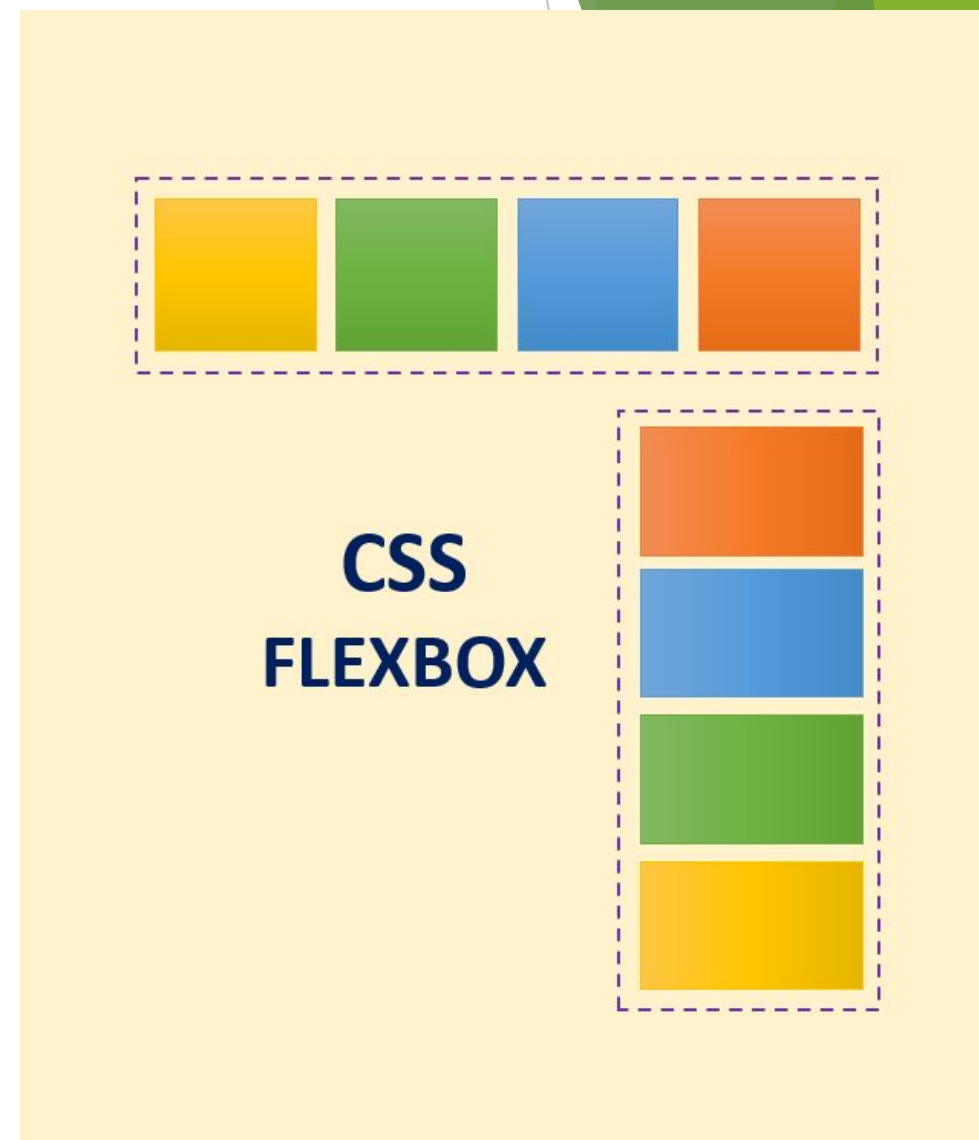
Aprendendo CSS 4

Aula 12.1: CSS flexbox

CSS Flexbox

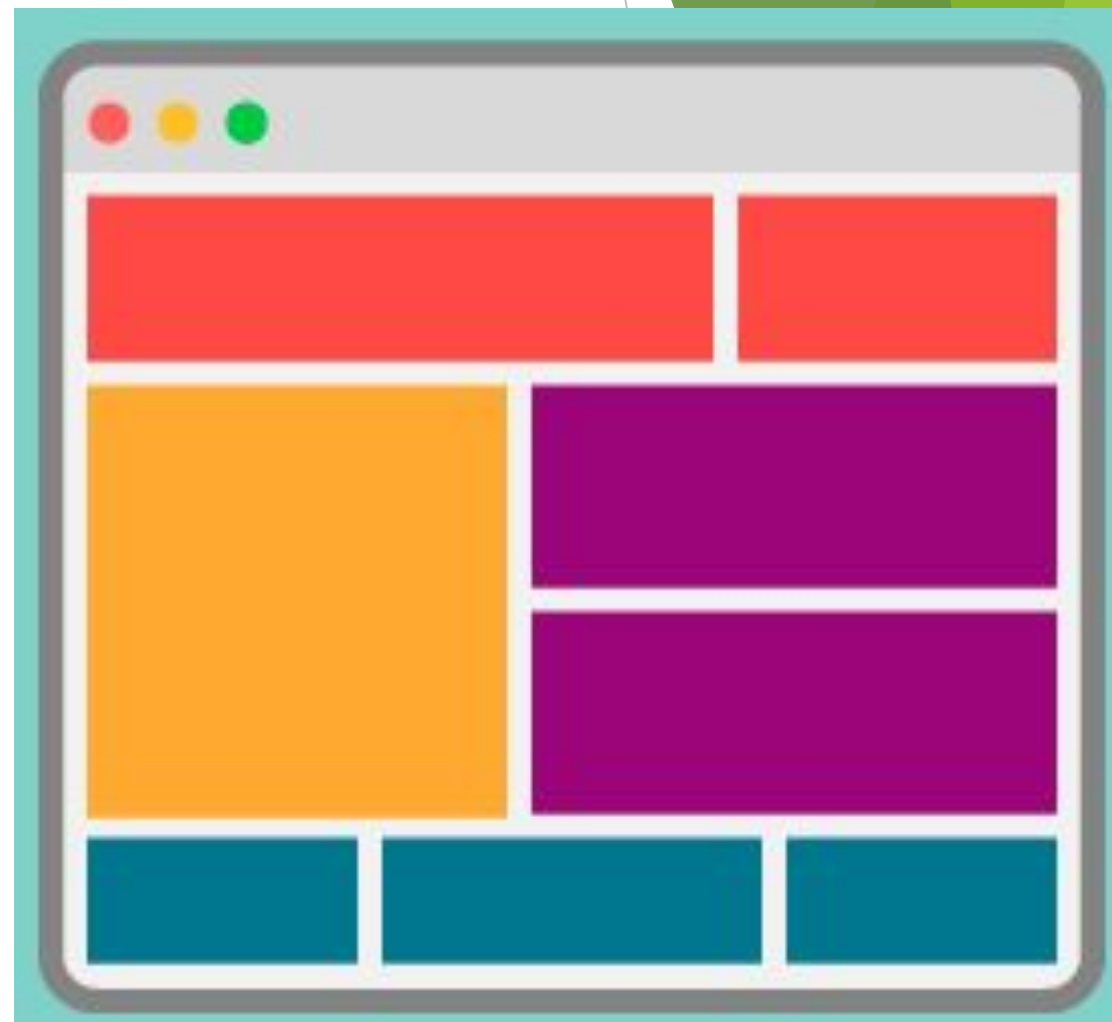
CSS: flexbox - o que é

- ❖ O Flexbox torna mais fácil criar **layouts flexíveis e responsivos** sem utilizar position ou float.
- ❖ Oferece uma maneira eficiente de **organizar, alinhar e distribuir** os itens dentro de um container, mesmo quando seus tamanhos são desconhecidos e/ou dinâmicos (daí o termo flexível).



CSS: flexbox - para que serve

- ❖ O container flex tem a capacidade de ajustar a sua altura/largura e ordem de seus itens para **preencher da melhor forma possível** o seu espaço disponível.
- ❖ Um container flex pode expandir os itens para ocupar o **espaço livre** ou reduzi-los para evitar overflow.

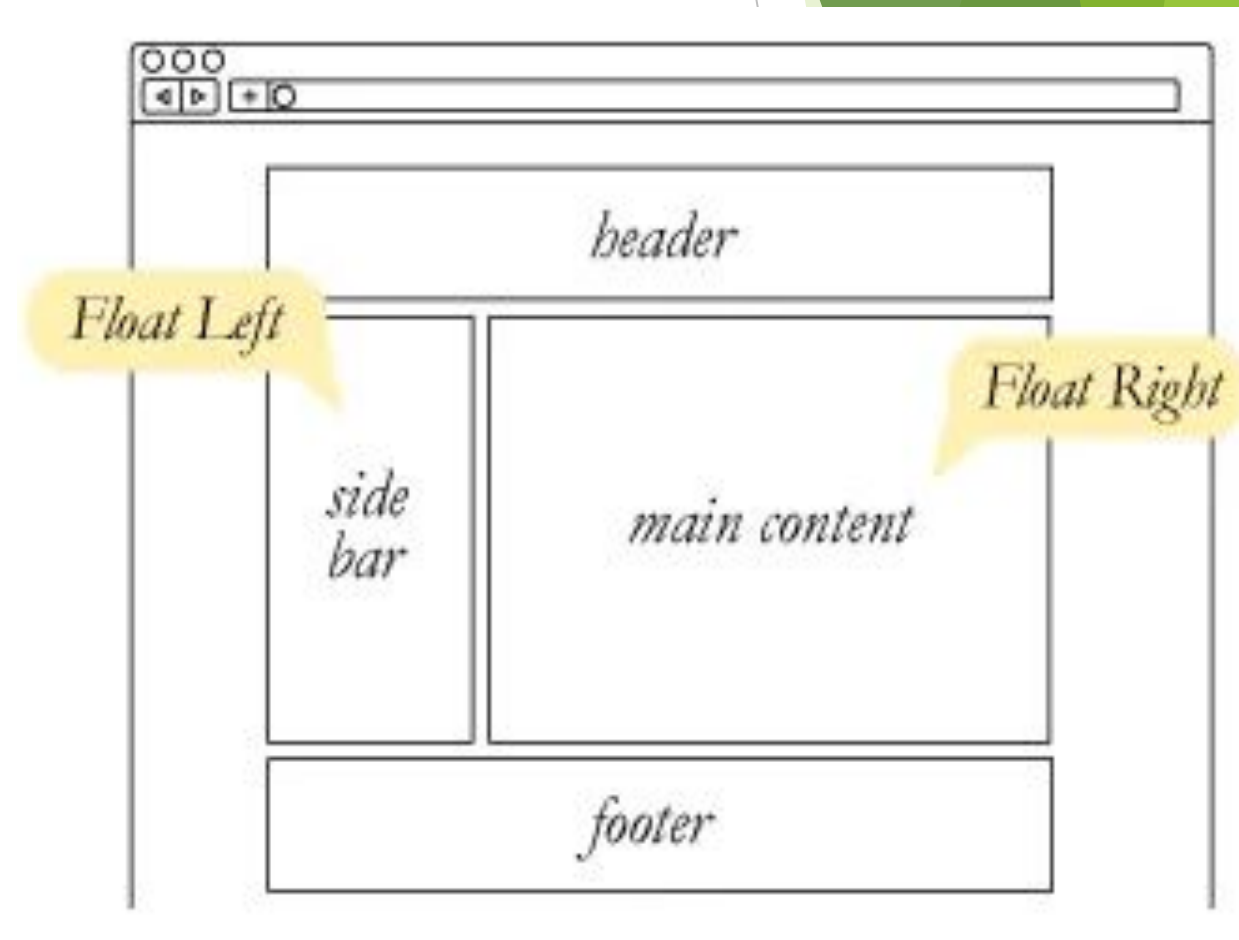


CSS: flexbox - o que é

- ❖ Ou seja, um container flex expande seus itens para **preencher** o espaço livre disponível ou os reduz para **evitar** que ultrapassem os limites do container.

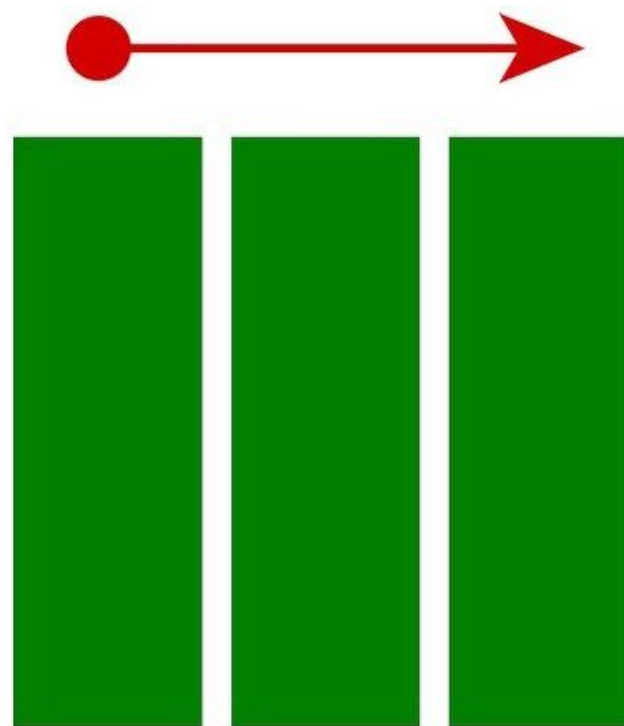
CSS: flexbox - histórico

- ❖ Antes do Flexbox, os desenvolvedores dependiam de técnicas como **float**, **inline-block** e até hacks com **tabelas** para criar layouts.
- ❖ Soluções que não foram pensadas originalmente para esse propósito e geravam muita complexidade e inconsistência.

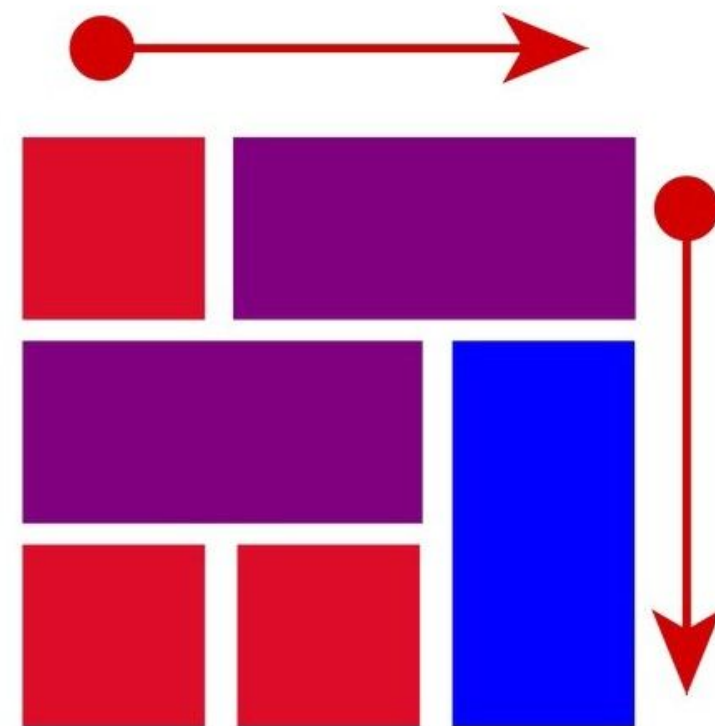


CSS: flexbox e grid

- ❖ E ele surgiu para resolver esses problemas.
- ❖ Posteriormente, ele foi complementado pelo CSS Grid, que atende melhor layouts **bidimensionais**, enquanto o Flexbox continua sendo a principal escolha para layouts **unidimensionais**.



Flexbox
One Dimensions



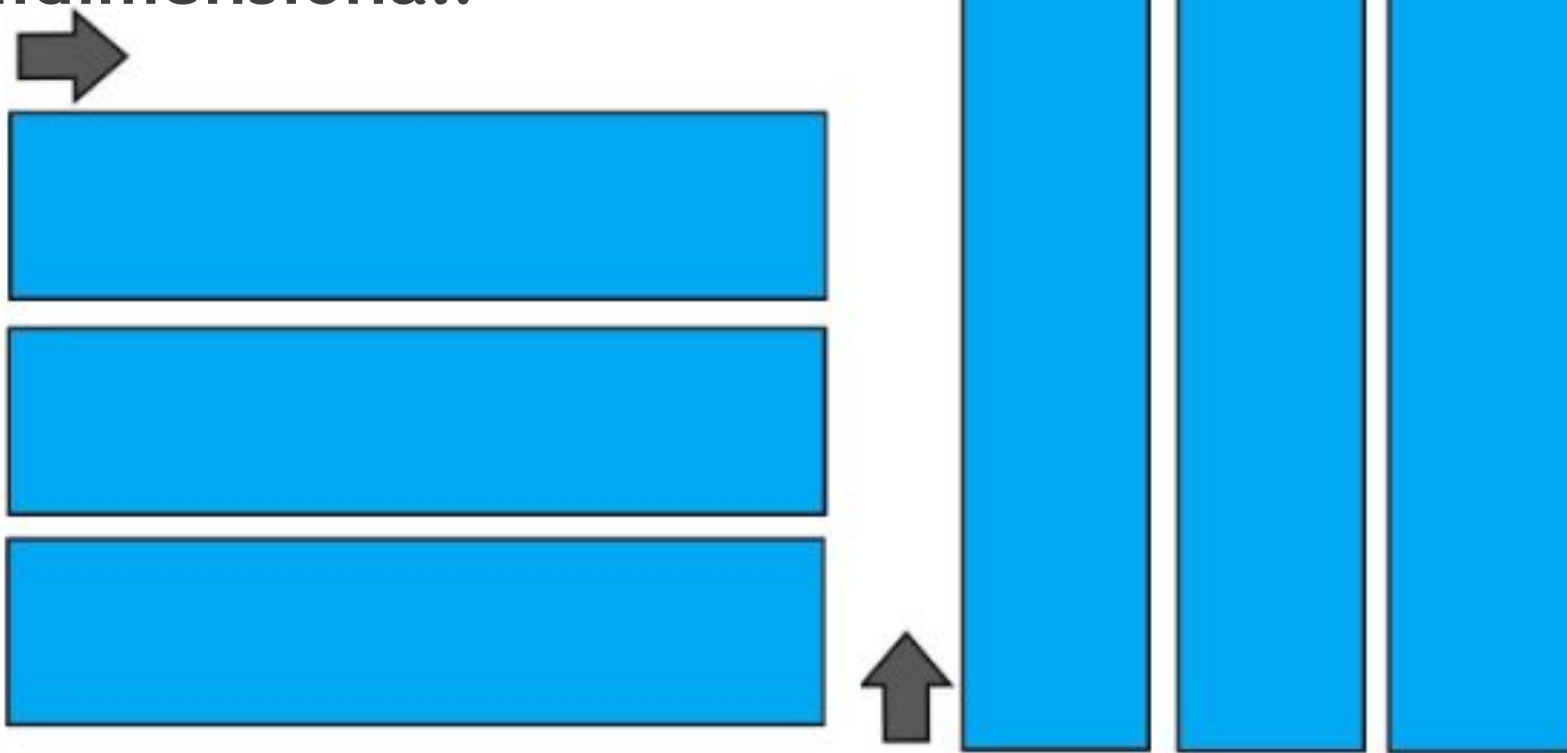
CSS Grids
Two Dimensions

CSS: flexbox e grid

- ❖ A diferença mais fundamental entre Grid e Flexbox está na forma como eles lidam com o **posicionamento dos elementos**.
- ❖ O Flexbox é **unidimensional**, trabalhando apenas uma direção por vez, ou uma linha (horizontal) ou uma coluna (vertical). Usado para layouts mais **simples**.
- ❖ O Grid é ideal para layouts **bidimensionais**, pois consegue gerenciar linhas e colunas ao mesmo tempo. Usado para layouts mais **complexos**.

CSS: flexbox - o que é

- ❖ O Flexbox, ou Flexible Box Layout, é um método de layout unidimensional.



Flexbox is one-dimensional

CSS: flexbox - conceitos

Propriedades do Contêiner:

- **flex-direction:** Direção dos itens;
- **flex-wrap:** Quebra de linha (nowrap, wrap, wrap-reverse);
- **justify-content:** Alinhamento no eixo principal (flex-start, flex-end, center, space-between, space-around, space-evenly);
- **align-items:** Alinhamento no eixo cruzado (stretch, flex-start, flex-end, center, baseline);
- **align-content:** Alinhamento de linhas no eixo cruzado quando há espaço extra;

Propriedades dos Itens:

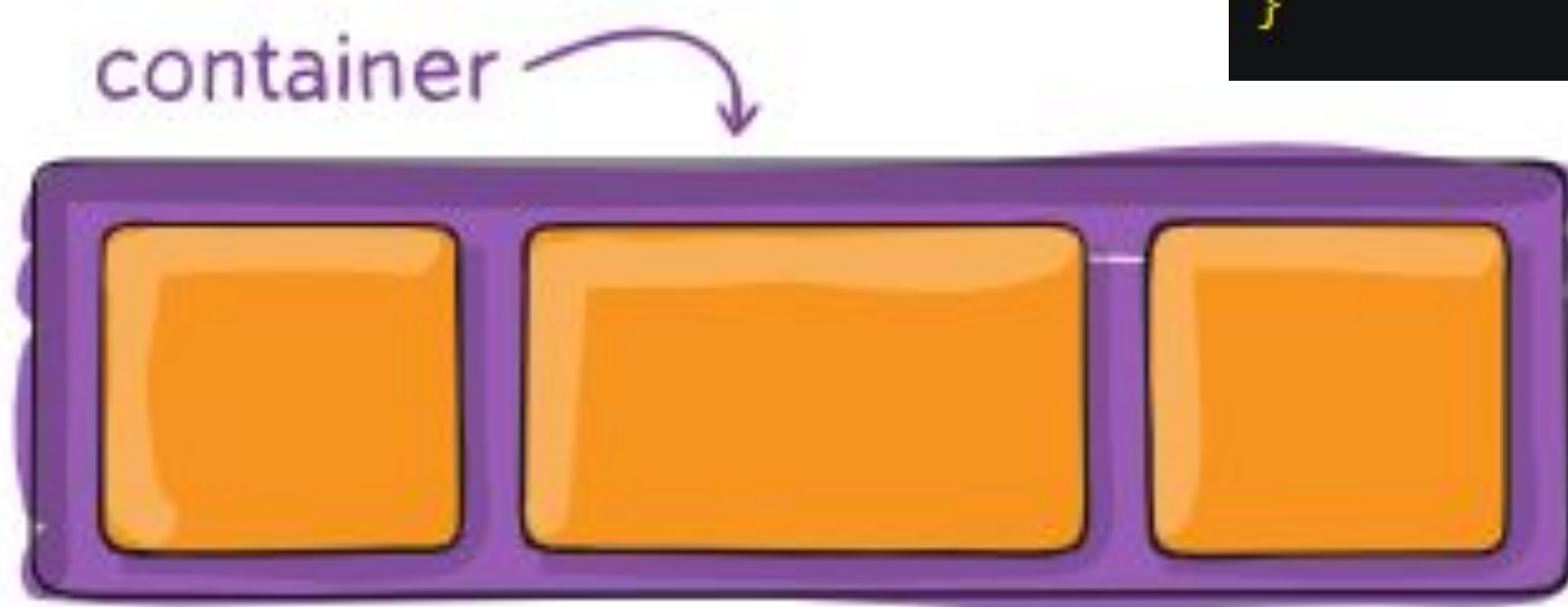
- **order:** Ordem de exibição dos itens;
- **flex-grow:** Capacidade de crescer para preencher espaço disponível;
- **flex-shrink:** Capacidade de encolher quando necessário;
- **flex-basis:** Tamanho inicial do item antes da distribuição de espaço;
- **flex:** Atalho para flex-grow, flex-shrink e flex-basis;
- **align-self:** Alinhamento individual de um item no eixo cruzado;

CSS: flexbox - conceitos

- ❖ **Flex Container:** O elemento que define a propriedade display como flex ou inline-flex.

```
/* flex-container */  
.products {  
  display: flex;  
  gap: 20px;  
  flex-direction: column;  
  flex-wrap: wrap;  
  /* quebra linha automaticamente */  
  padding: 20px;  
}
```

Fonte: autora, 2026



Fonte: Coyier, 2026

CSS: flexbox - conceitos

- ❖ **Flex Items:** Os filhos diretos do container flex.

```
/* flex-items */  
.product-card {  
  flex: 1 1 30%;  
  /* ocupa ~30% da tela */  
  border: 1px solid #ccc;  
  padding: 10px;  
  text-align: center;  
}
```

Fonte: autora, 2026



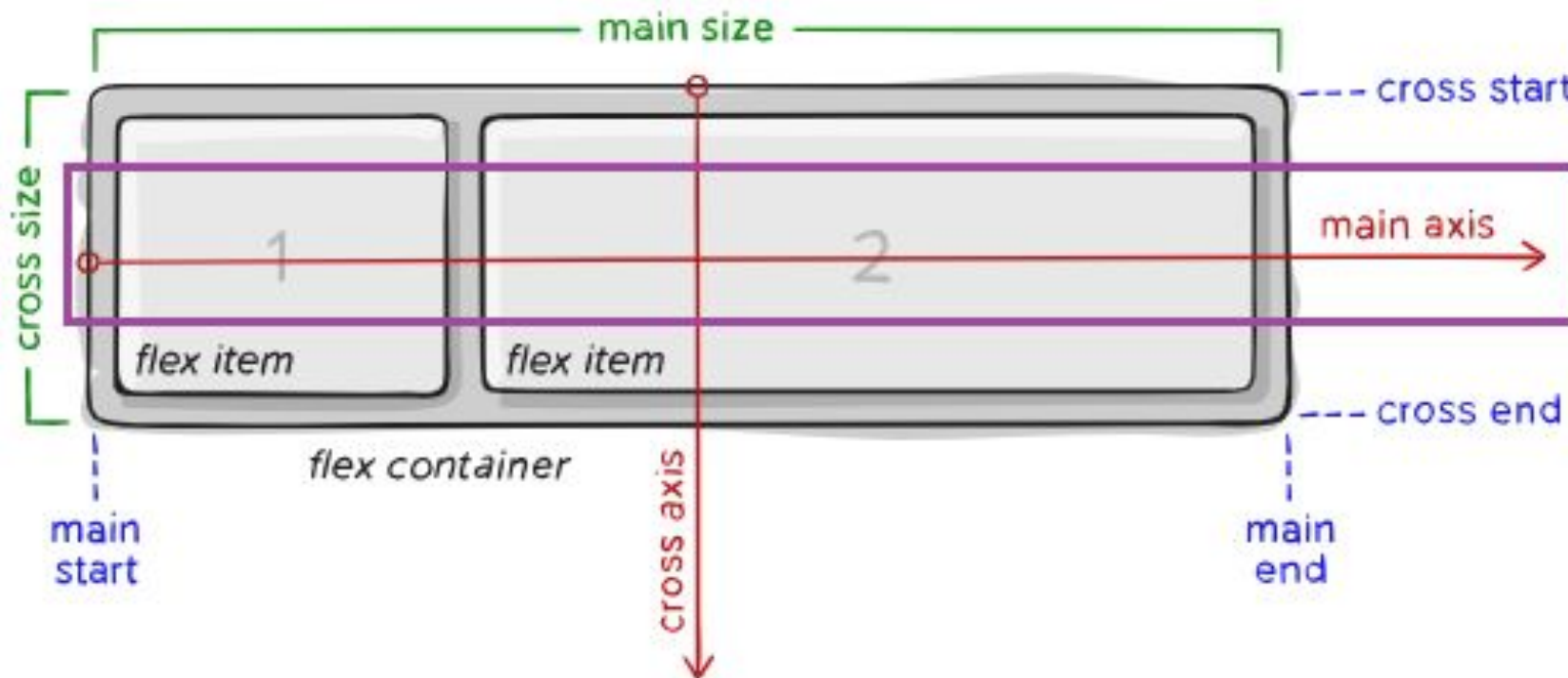
Fonte: Coyier, 2026

CSS: flexbox - conceitos

- ❖ O layout com Flexbox é baseado nas chamadas “flex-flow directions” (direções de fluxo flexível).
- ❖ Nele, os itens são organizados seguindo **dois eixos principais**: o eixo principal (main axis) e o eixo transversal (cross axis).

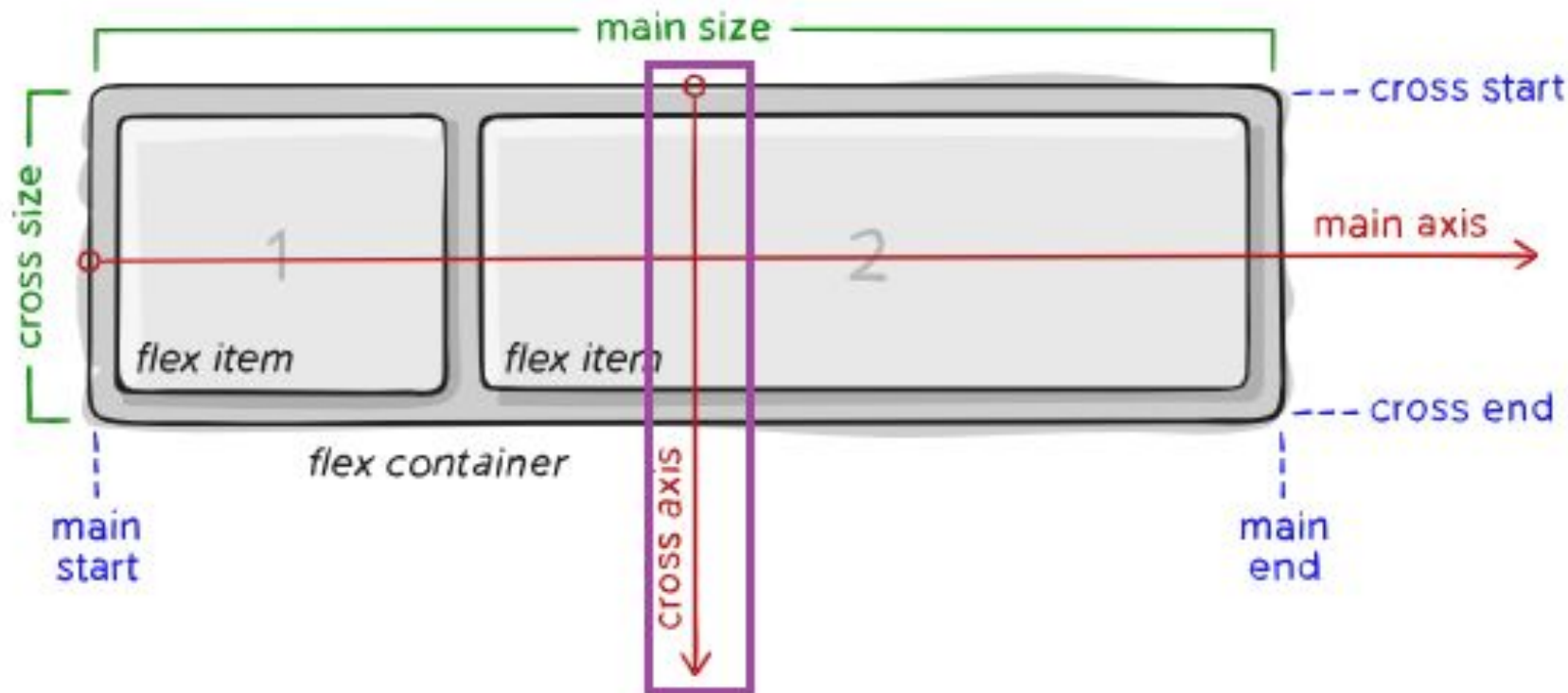
CSS: flexbox - conceitos

- ❖ **Eixo Principal (Main Axis):** É o eixo principal ao longo do qual os itens flex são organizados. *Importante: ele não é necessariamente horizontal, pois depende da propriedade **flex-direction**.*



CSS: flexbox - conceitos

- ❖ **Eixo transversal (cross axis):** É o eixo perpendicular ao eixo principal. Sua direção depende da orientação do main axis.



Propriedades para container

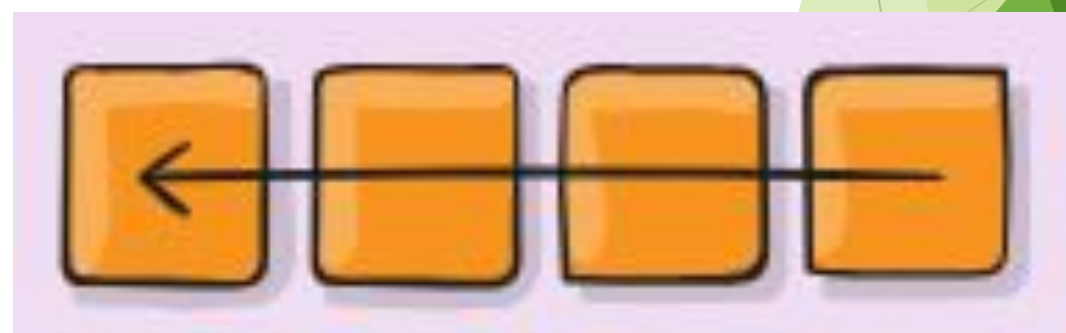
CSS: flexbox - conceitos

- ❖ **Flex-direction:** define qual vai ser o eixo principal (main axis), e a direção que seus itens serão ordenados no container.

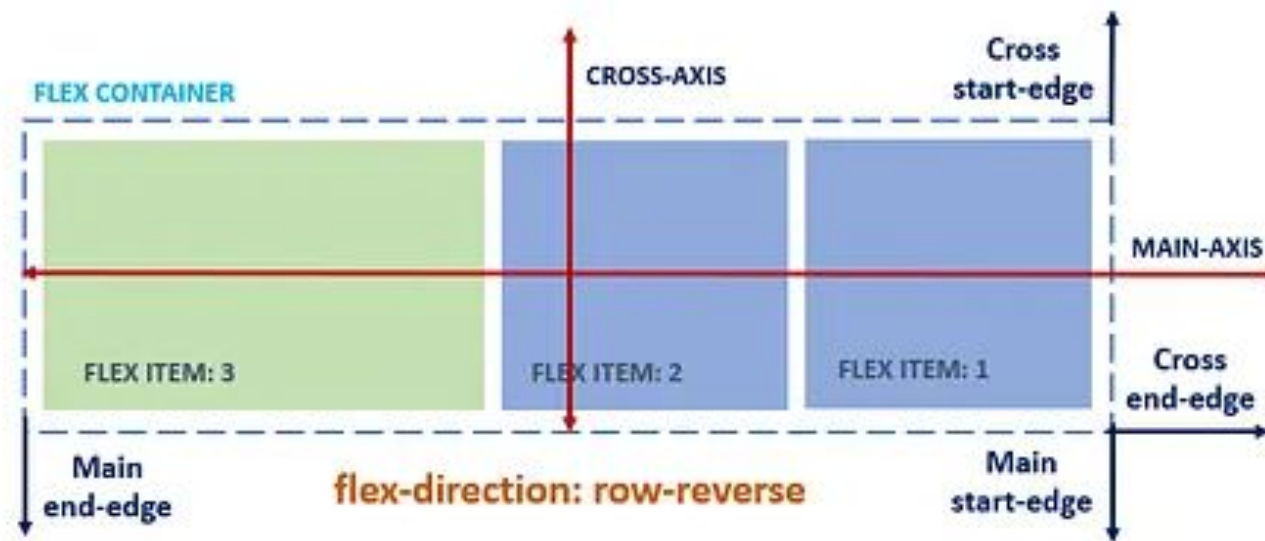
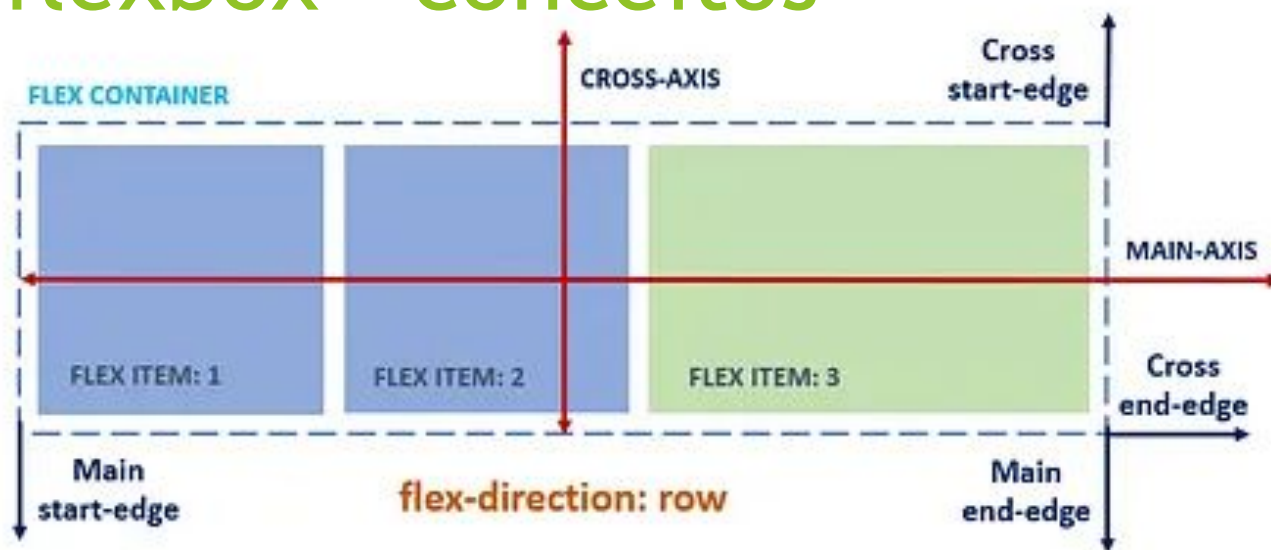
```
.products {  
  flex-direction: row;  
}
```



```
.products {  
  flex-direction: row-reverse;  
}
```

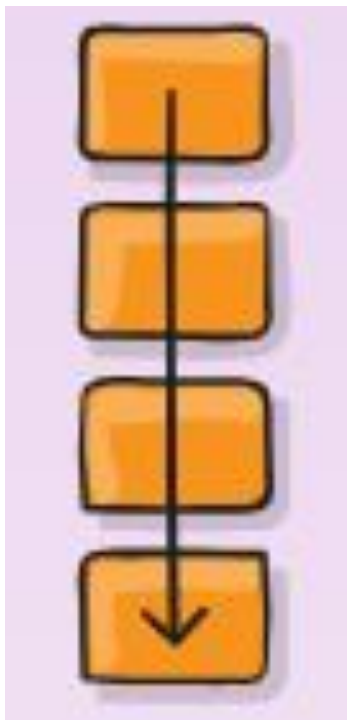


CSS: flexbox - conceitos

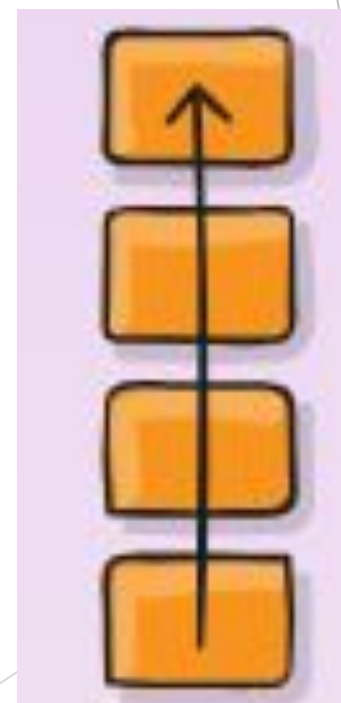


CSS: flexbox - conceitos

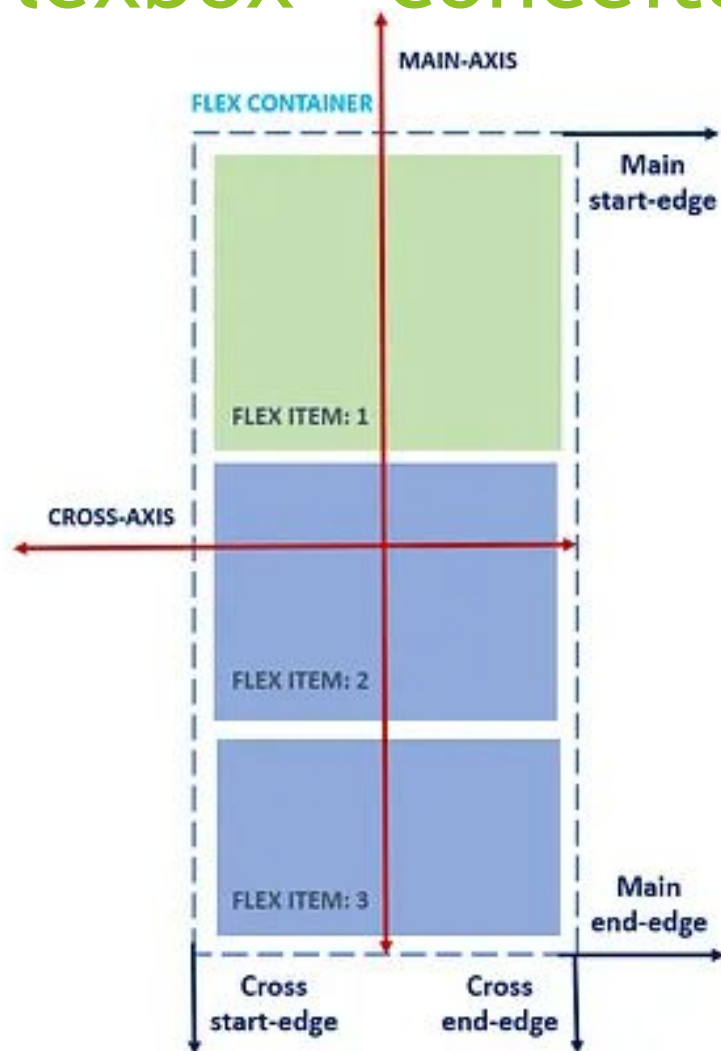
```
.products {  
  flex-direction: column;  
}
```



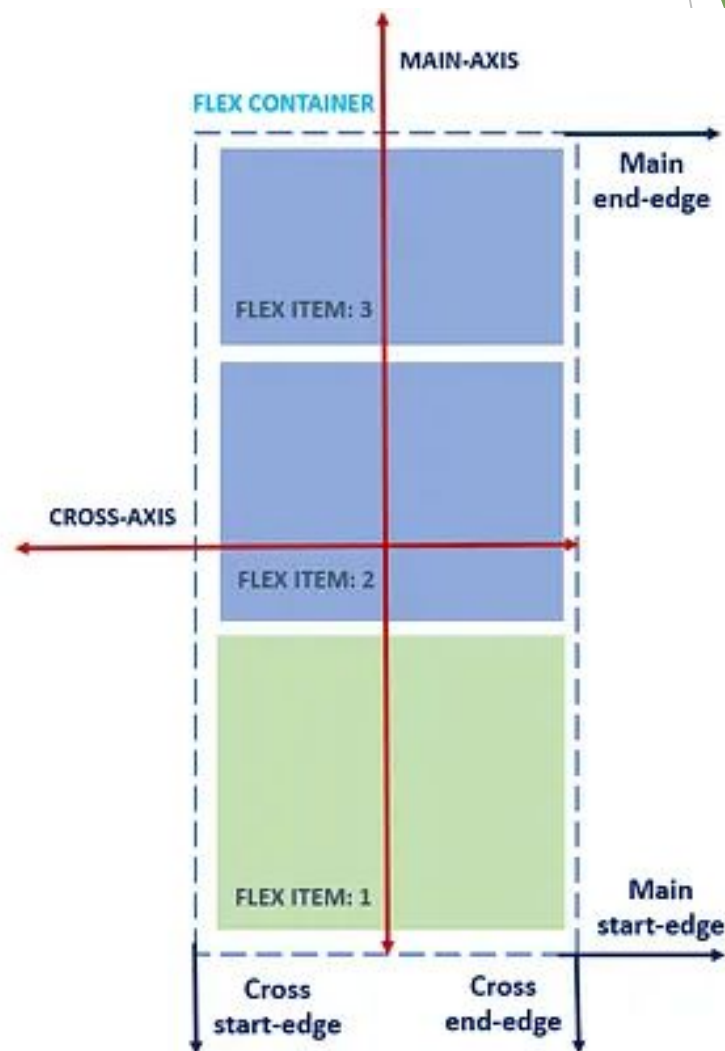
```
.products {  
  flex-direction: column-reverse;  
}
```



CSS: flexbox - conceitos



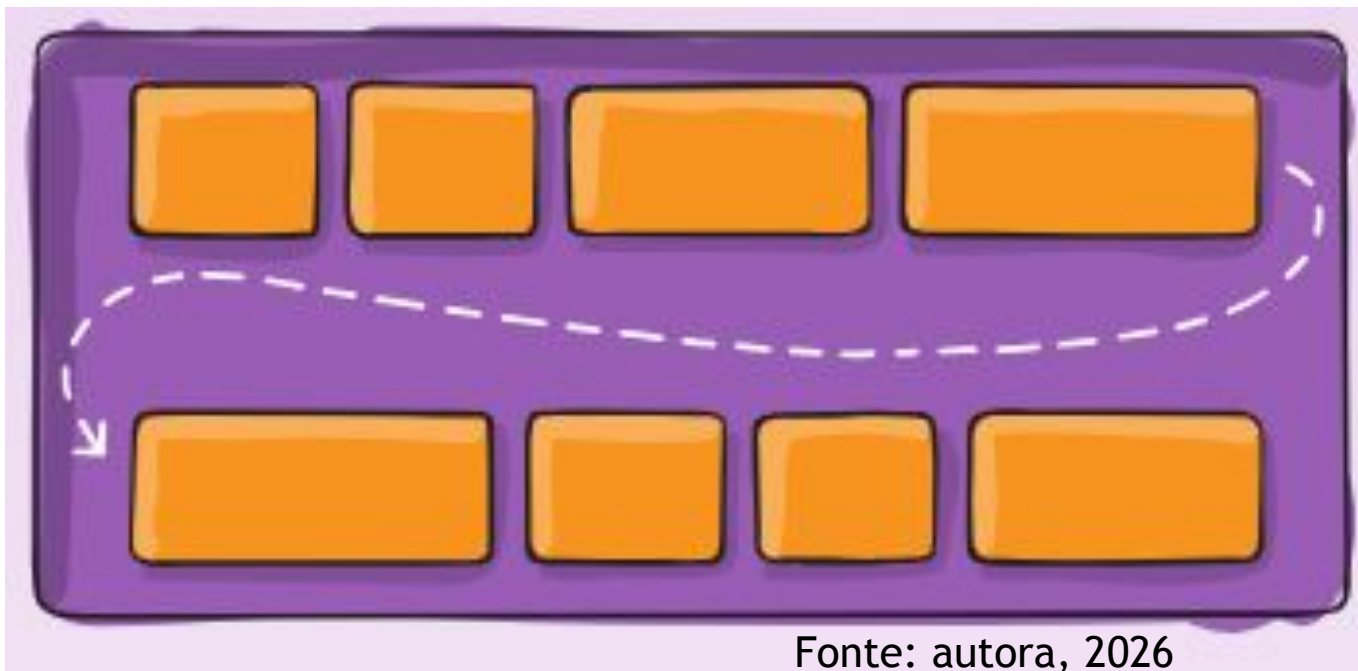
`flex-direction: column`



`flex-direction: column-reverse`

CSS: flexbox - propriedades

- ❖ **Flex-wrap:** os itens flex tentarão se ajustar em uma única linha. O wrap permite mudar esse comportamento para que os itens **quebrem** para **múltiplas linhas** conforme necessário.



Fonte: autora, 2026

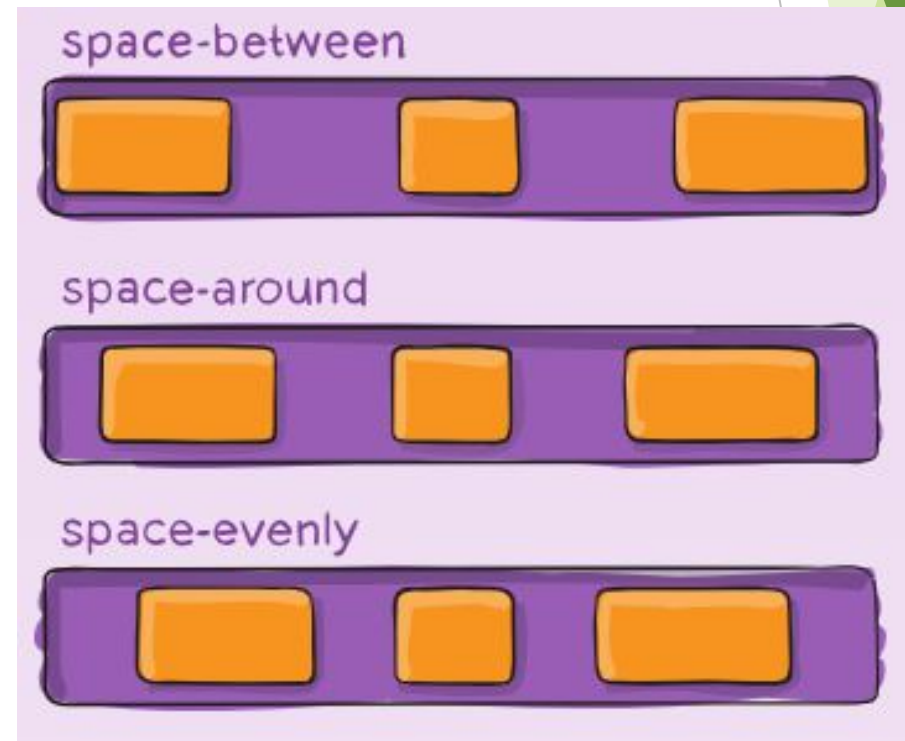
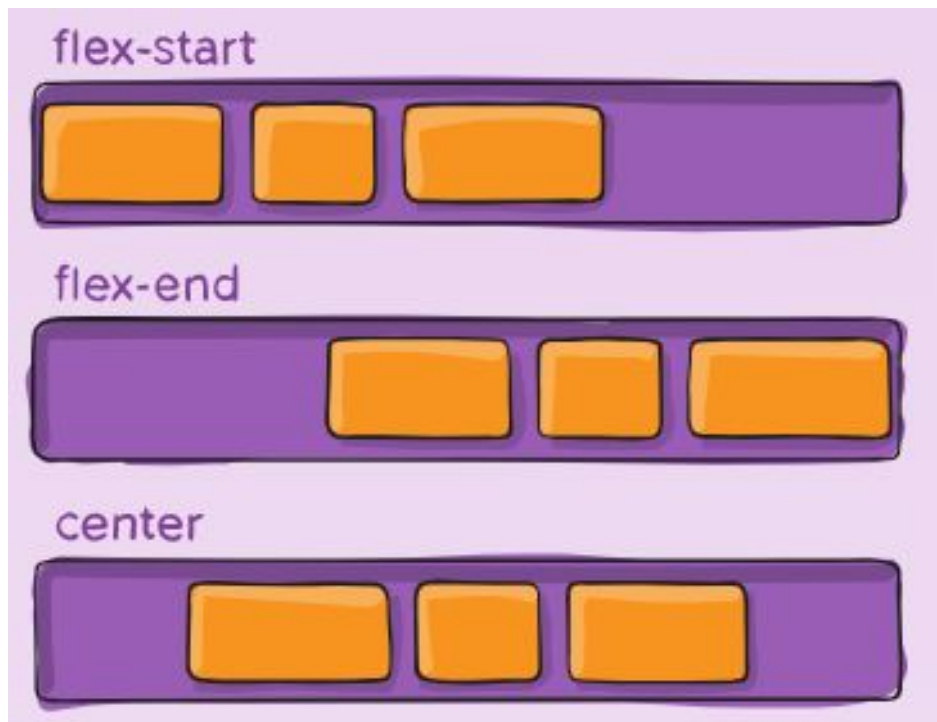
```
.products {  
  flex-wrap: nowrap;  
}
```

```
.products {  
  flex-wrap: wrap;  
}
```

```
.products {  
  flex-wrap: wrap-reverse;  
}
```

CSS: flexbox - propriedades

- ❖ **Justify-content:** define o alinhamento ao longo do eixo principal (main axis).



CSS: flexbox - propriedades

```
.products {  
  justify-content: space-between;  
}
```



```
.products {  
  justify-content: flex-start;  
}
```

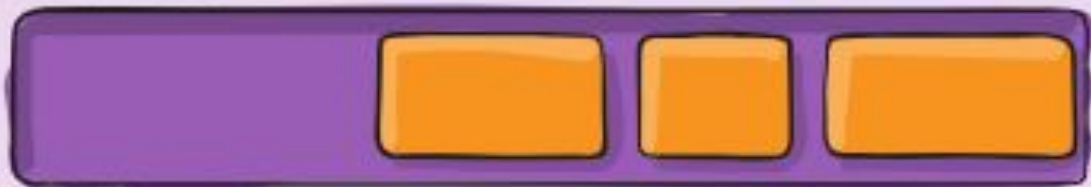


- ❖ **space-between:** os itens são distribuídos uniformemente; o primeiro fica no início e o último no final

CSS: flexbox - propriedades

```
.products {  
  justify-content: flex-end;  
}
```

flex-end



```
.products {  
  justify-content: space-around;  
}
```

space-around



❖ **space-around:** os itens são distribuídos com espaço ao redor de cada um.

Visualmente, os espaços não são iguais, pois cada item tem espaço dos dois lados, o primeiro item terá metade do espaço na borda em comparação ao espaço entre itens

CSS: flexbox - propriedades

```
.products {  
  justify-content: center;  
}
```

center



```
.products {  
  justify-content: space-evenly;  
}
```

space-evenly

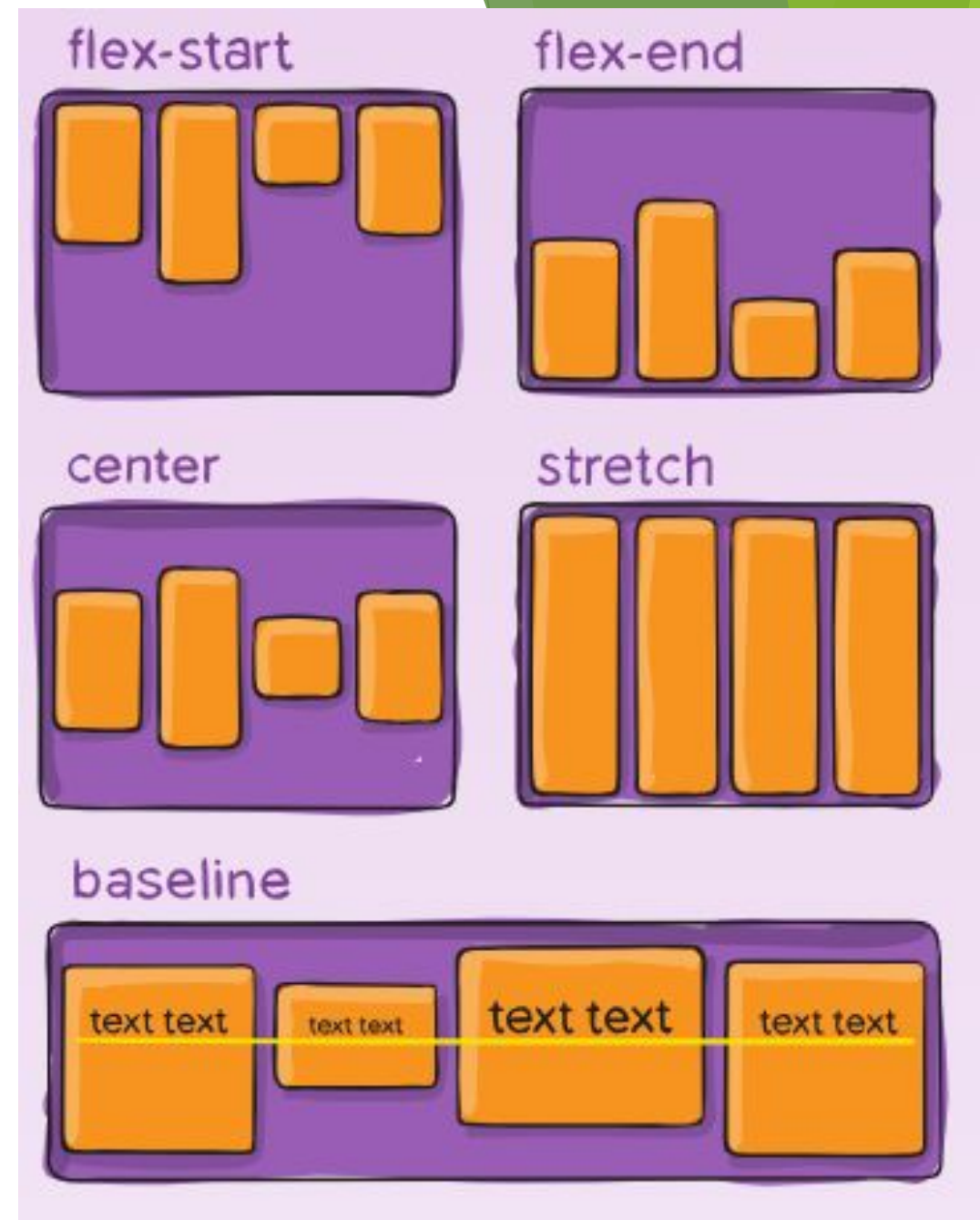


- ❖ **space-evenly:** os itens são distribuídos de forma que o espaço entre eles e nas bordas seja exatamente igual

CSS: flexbox - propriedades

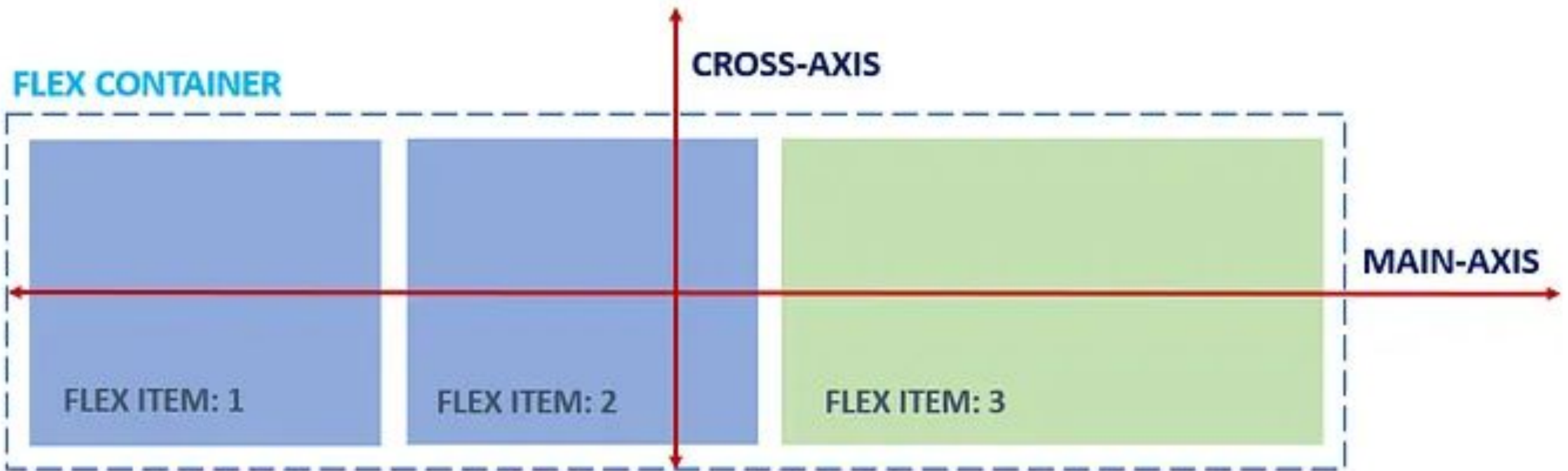
❖ Align-items:

Define o comportamento padrão de como os itens flex são organizados ao longo do eixo transversal (**cross axis**) na linha atual.

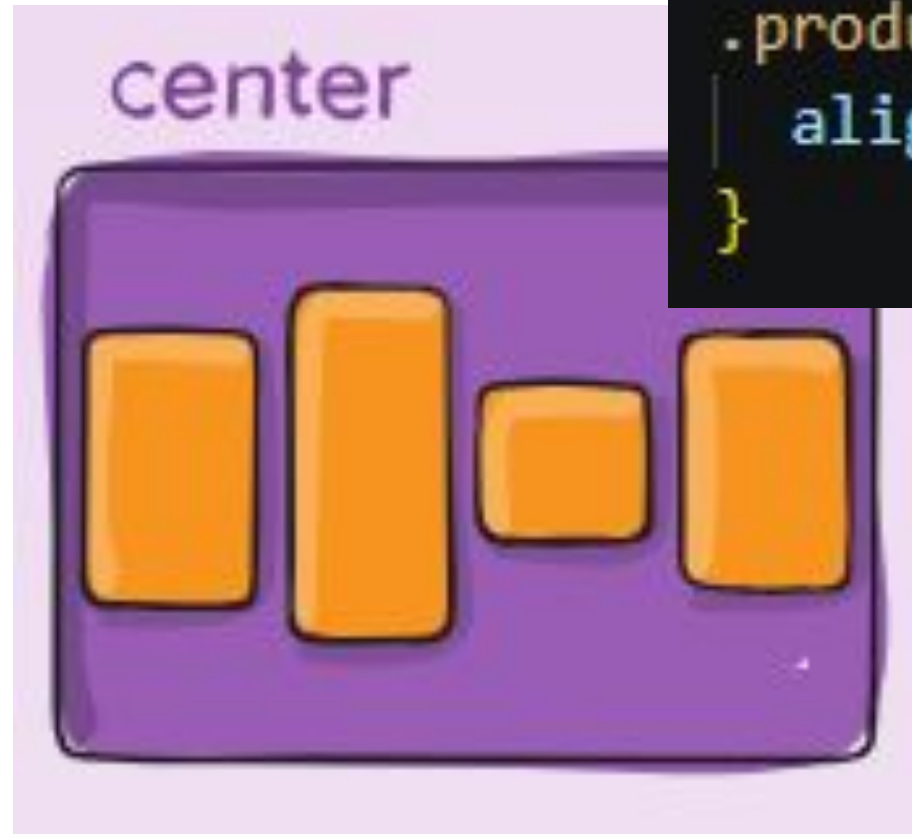


CSS: flexbox - propriedades

- ❖ Pense nele como a versão do justify-content, porém para o **eixo transversal** (perpendicular ao eixo principal).



CSS: flexbox - propriedades



```
.products {  
  align-items: center;  
}
```

- ❖ **center:** os itens são centralizados no eixo transversal

CSS: flexbox - propriedades



```
.products {  
  align-items: stretch;  
}
```

- ❖ **stretch:** os itens se esticam para preencher o container (respeitando min-width e max-width)

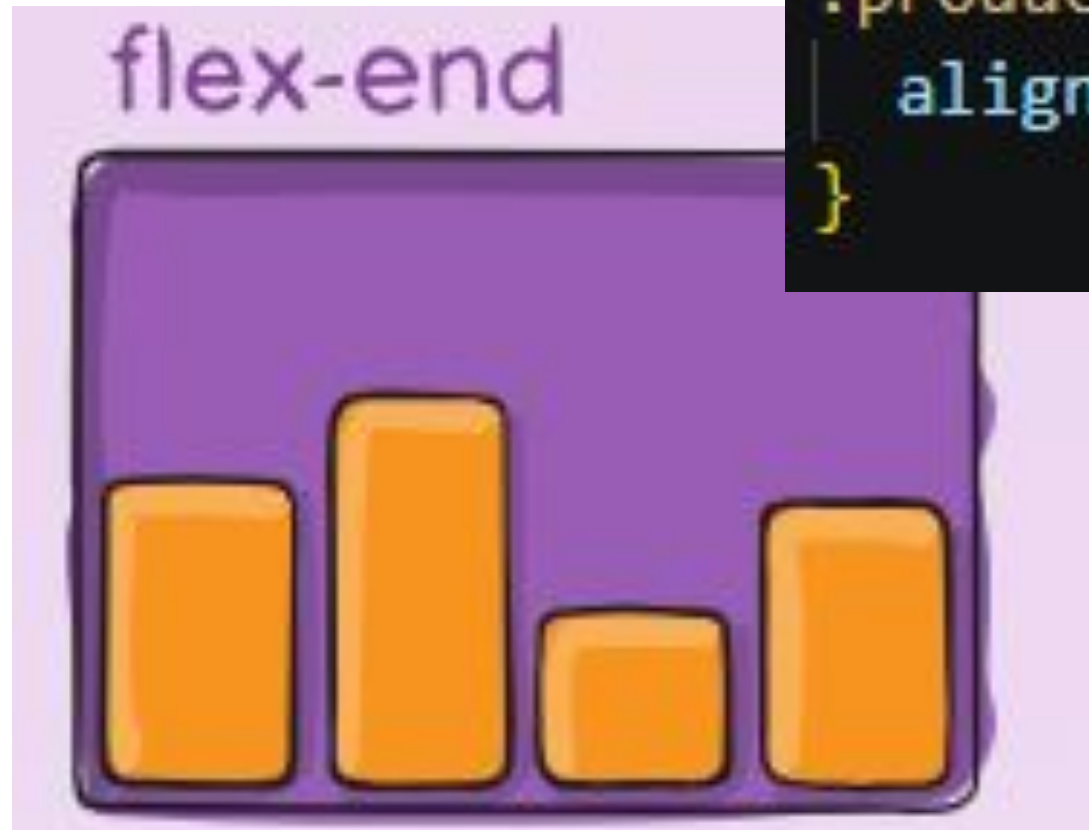
CSS: flexbox - propriedades



```
.products {  
  align-items: flex-start;  
}
```

- ❖ **flex-start:** os itens são posicionados no início do eixo transversa.

CSS: flexbox - propriedades

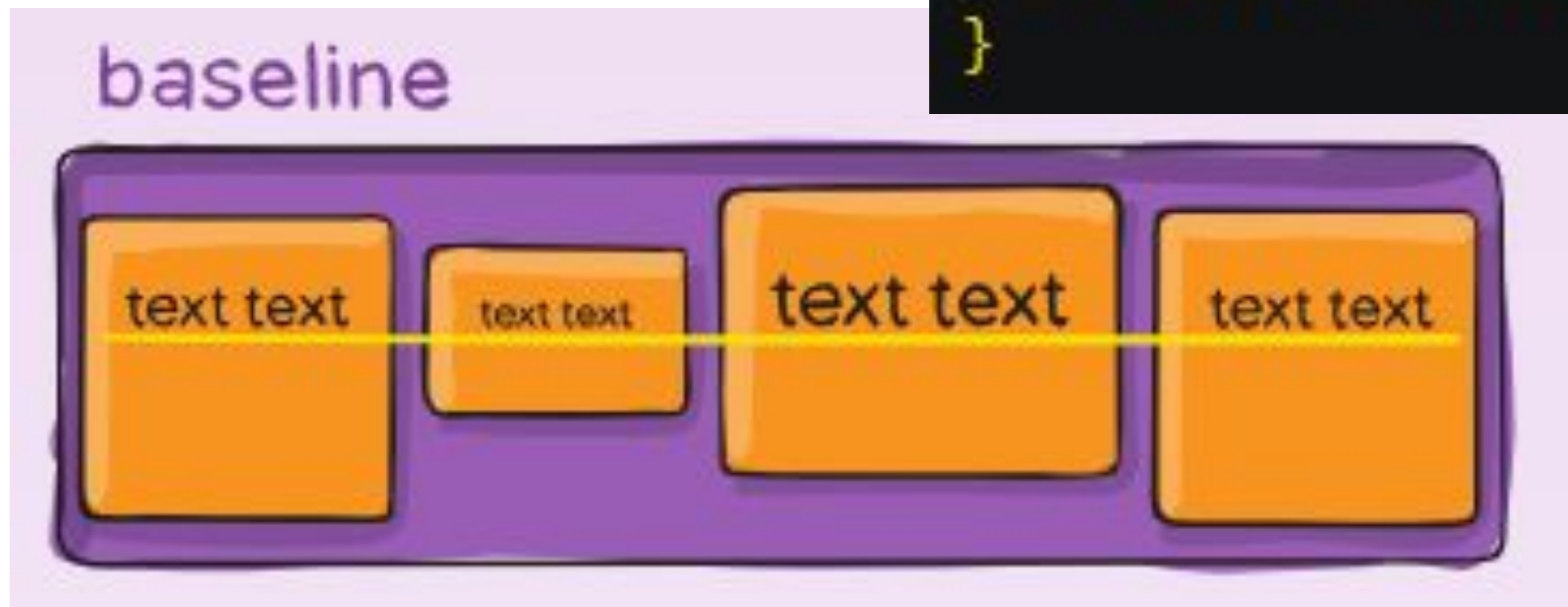


```
.products {  
  align-items: flex-end;  
}
```

- ❖ **flex-end:** os itens são posicionados no final do eixo transversal.

CSS: flexbox - propriedades

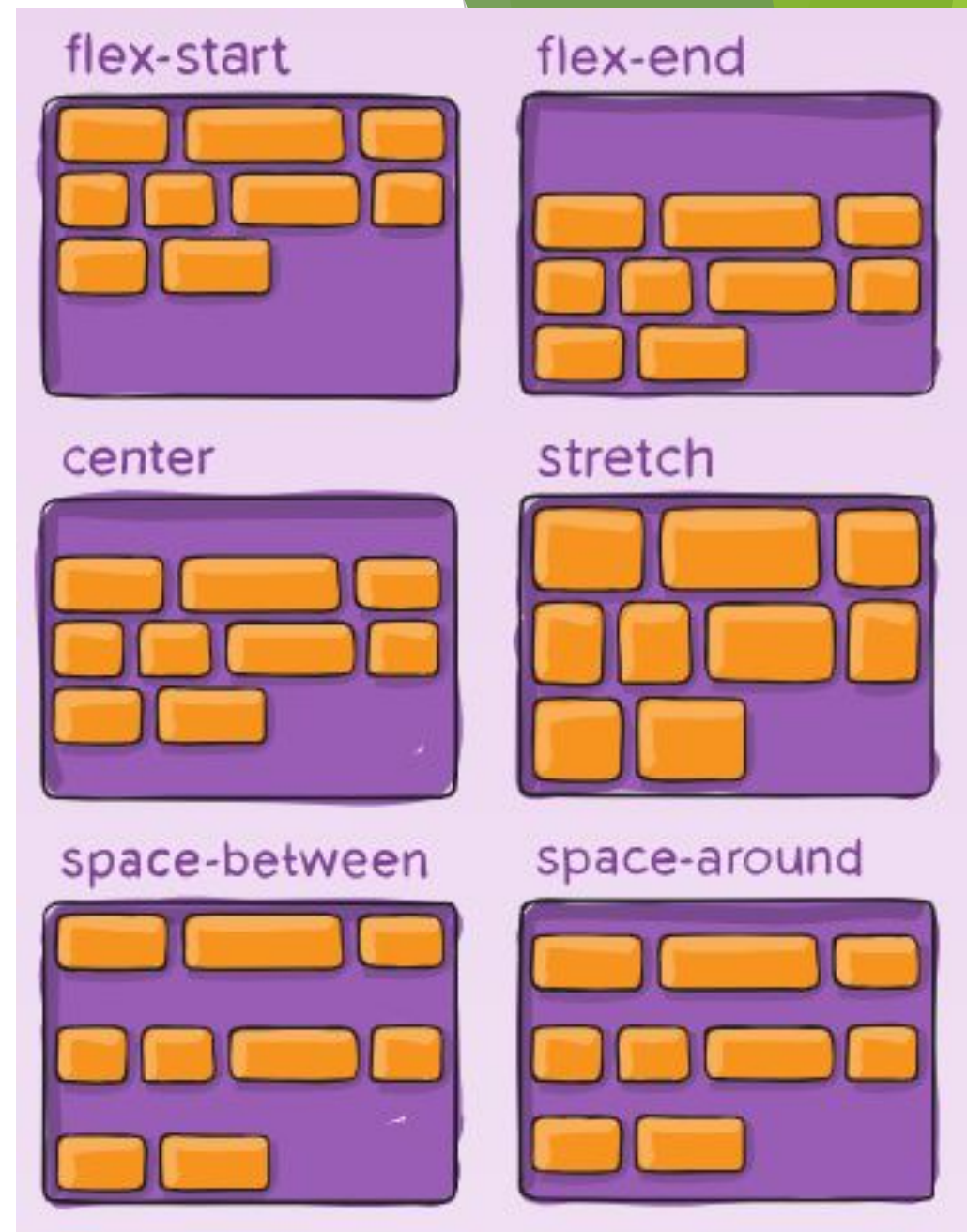
```
.products {  
  align-items: baseline;  
}
```



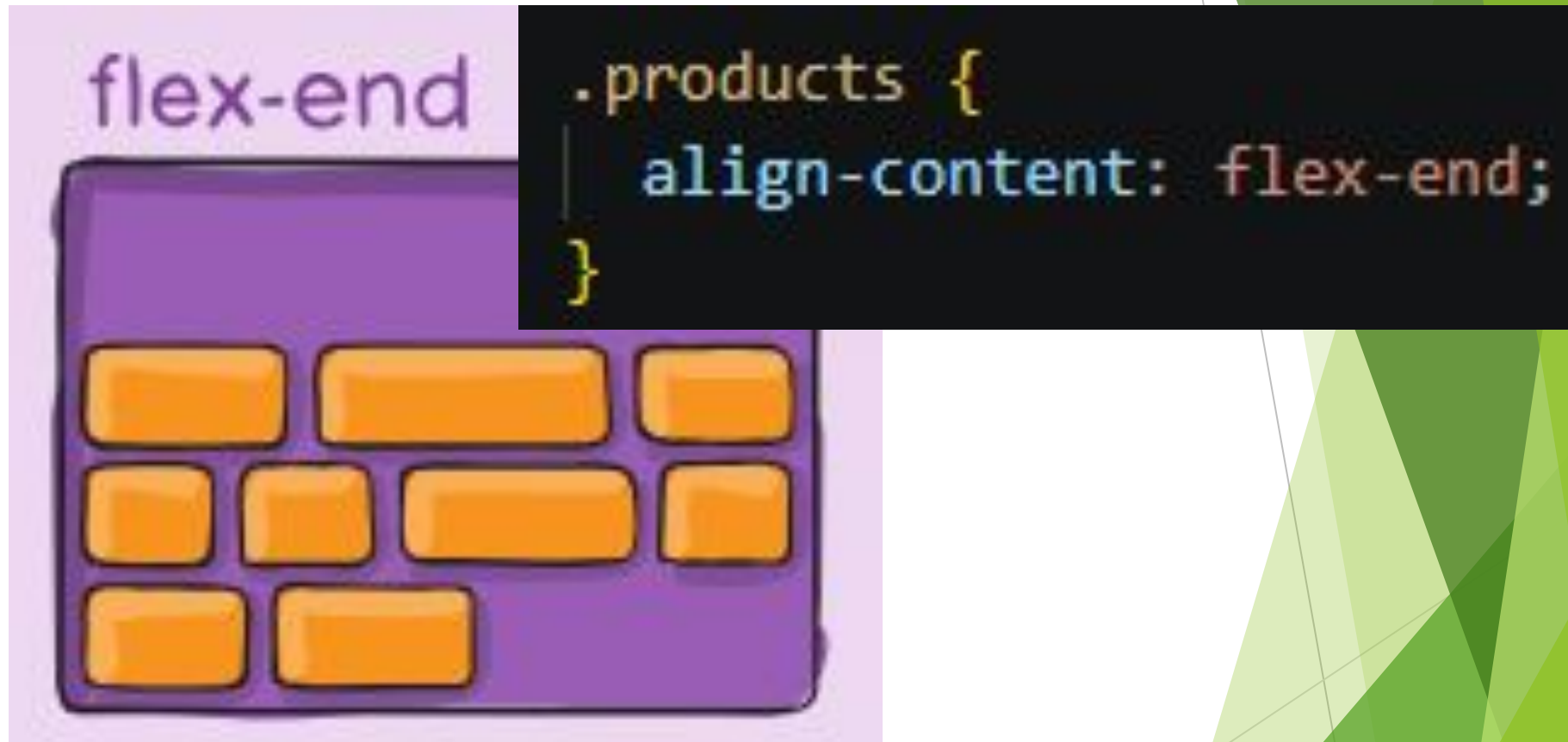
CSS: flexbox - propriedades

❖ Align-content:

Alinha os itens dentro de um container flex quando há **espaço extra** no eixo transversal (cross axis), de forma semelhante a como o justify-content alinha os itens no eixo principal.



CSS: flexbox - propriedades



- ❖ **flex-end:** as linhas são agrupadas no final do container.

CSS: flexbox - propriedades



```
.products {  
  align-content: flex-start;  
}
```

- ❖ **flex-start:** as linhas são agrupadas no início do container.

CSS: flexbox - propriedades



```
.products {  
  align-content: center;  
}
```

- ❖ **center:** as linhas ficam centralizadas no container.

CSS: flexbox - propriedades



```
.products {  
  align-content: stretch;  
}
```

- ❖ **space-evenly:** os itens são distribuídos de forma que o espaço entre eles e nas bordas seja exatamente igual.

CSS: flexbox - propriedades



```
.products {  
  align-content: space-between;  
}
```

- ❖ **space-between:** as linhas são distribuídas uniformemente, a primeira fica no início e a última no final.

CSS: flexbox - propriedades

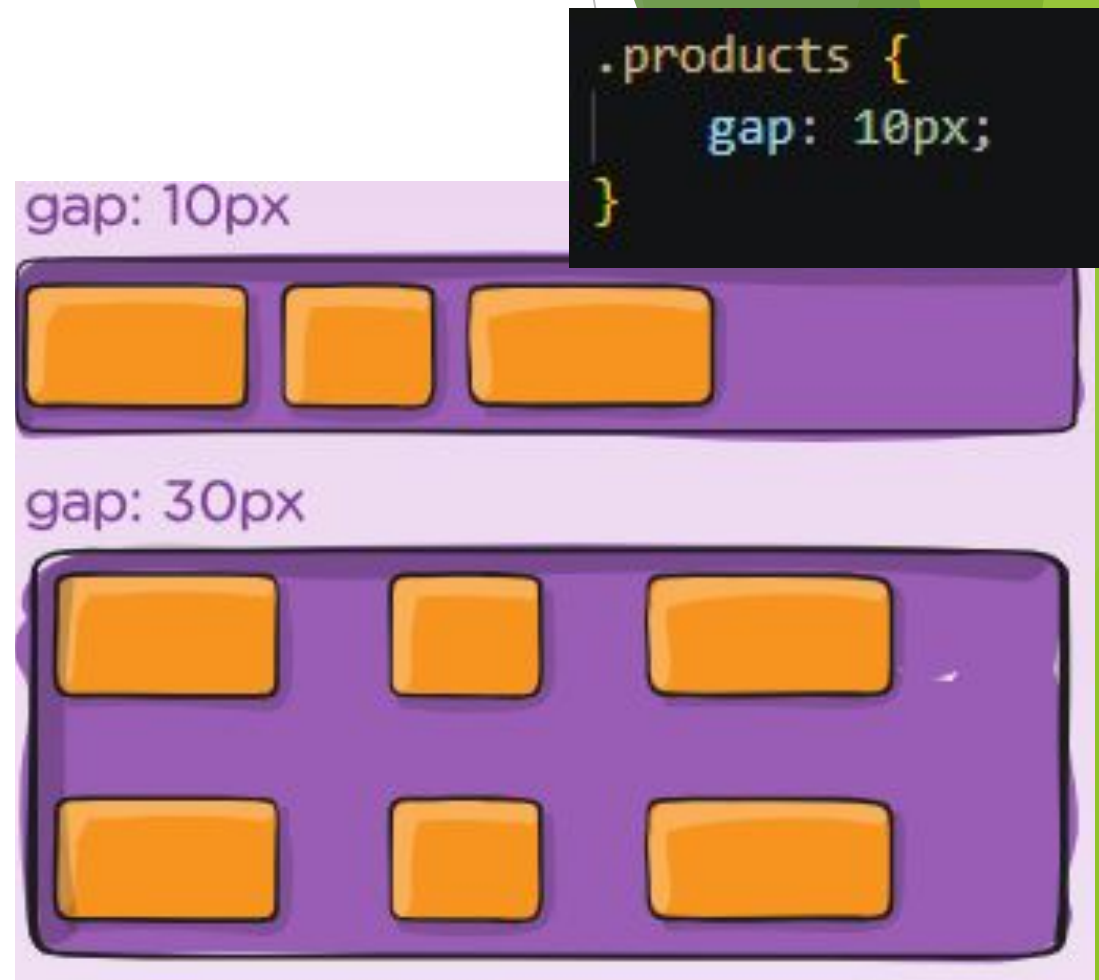


```
.products {  
  align-content: space-around;  
}
```

- ❖ **space-around:** as linhas são distribuídas com espaço igual ao redor de cada uma.

CSS: flexbox - propriedades

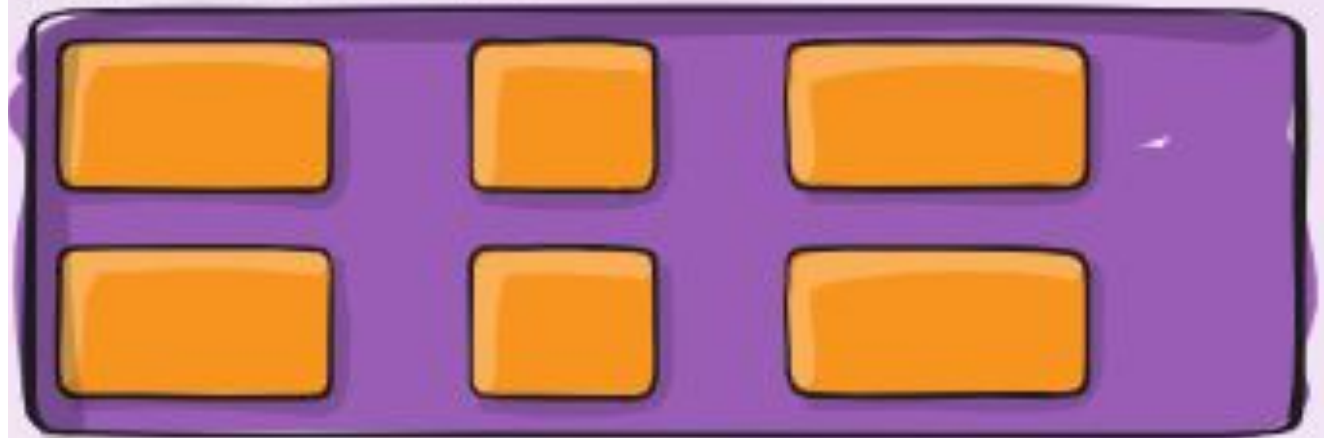
- ❖ **Gap:** controla explicitamente o espaço entre os itens flex. Esse espaçamento é aplicado apenas entre os itens, e não nas bordas externas do container.
- ❖ Se outro estilo (como justify-content) já cria um espaço maior, o gap é ignorado. Se o espaço for menor, o gap é aplicado.



CSS: flexbox - propriedades

- ❖ **Gap:** Pode ser definido também espaçamentos diferentes para cada orientação, com as propriedades *row-gap* e *column-gap*, ou definição diretamente com múltiplos valores no *gap*.

gap: 10px 30px



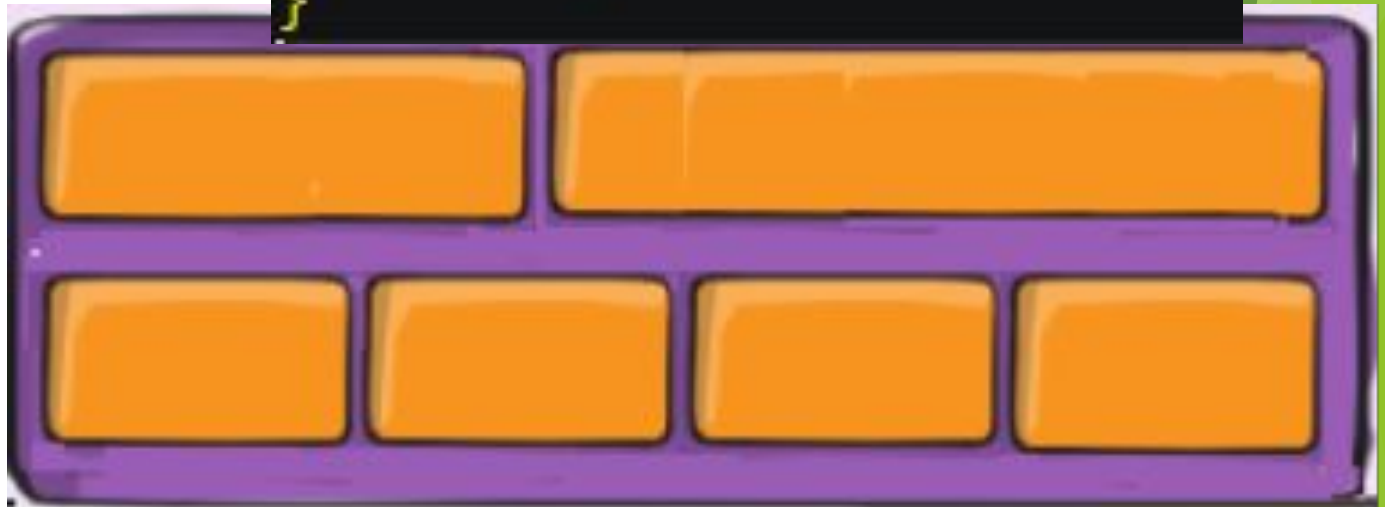
```
.products {  
  /* gap: 10px 30px; */  
  row-gap: 10px;  
  column-gap: 30px;  
}
```

Propriedades para elementos

CSS: flexbox - propriedades

- ❖ **Flex-grow:** Define o quanto o elemento filho irá se expandir para ocupar os espaços em branco do layout.

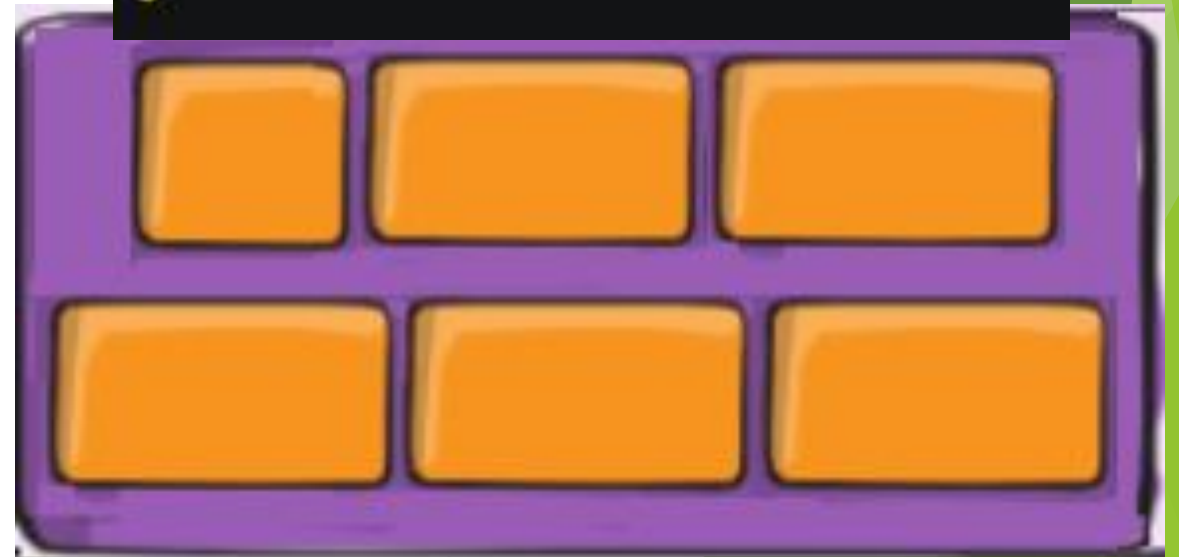
```
.product-card {  
  flex-grow: 1;  
}  
  
.product-card.second-item {  
  flex-grow: 2;  
}
```



CSS: flexbox - propriedades

- ❖ **Flex-shrink:** Define o quanto o elemento filho irá se comprimir quando diminuir a largura total do layout.

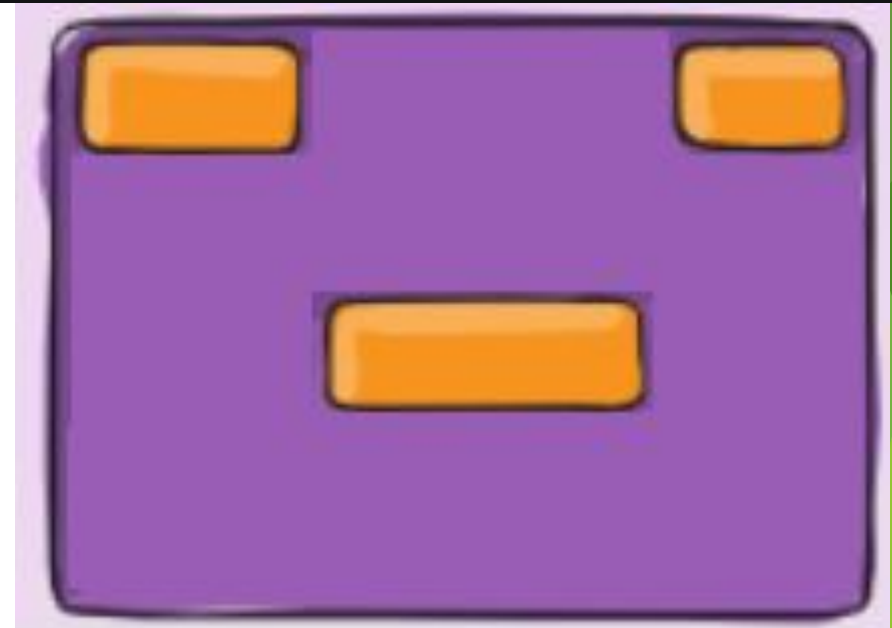
```
.product-card {  
  flex-shrink: 1;  
}  
  
.product-card.second-item {  
  flex-shrink: 2;  
}
```



CSS: flexbox - propriedades

- ❖ **Align-self:** aplica uma **direção de alinhamento** no elemento filho de acordo com a **orientação** do layout.
- ❖ **Valores:** flex-start, flex-end, baseline, center, stretch.

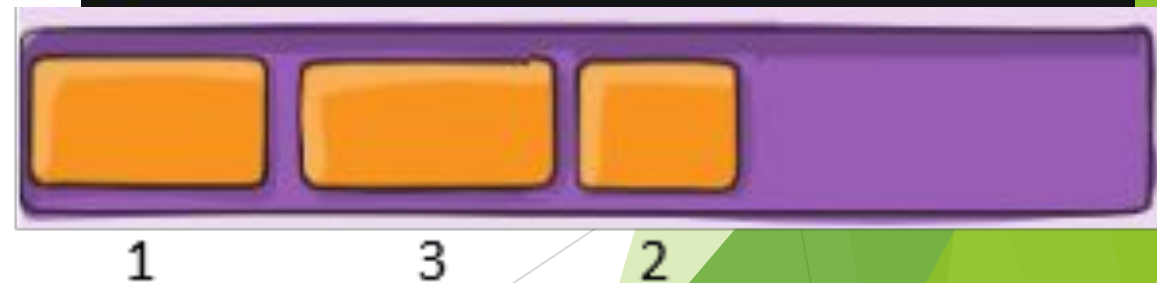
```
.product-card {  
  align-self: flex-start; /* valor padrão */  
}  
  
.product-card.second-item {  
  align-self: center;  
}
```



CSS: flexbox - propriedades

- ❖ **Order:** altera a ordem de apresentação do elemento filho de acordo com a **orientação** do layout.
- ❖ **Valores:** os menores valores aparecem primeiro e os maiores ao final.

```
.product-card {  
  order: 0; /* valor padrão */  
}  
  
.product-card.second-item {  
  order: 2;  
}
```



Aplicação no projeto

CSS: flexbox

- ❖ Desta forma, o flexbox consegue trabalhar com colunas ou linhas, mas **não** com ambos ao mesmo tempo.

```
.products {  
  flex-direction: column;  
}
```

Samsung



R\$ 2.000,00

R\$ 200,00 em 10x

[Ver mais](#)

Samsung



LG



R\$ 5.000,00

500,00 em 10x

[Ver mais](#)

Philco



R\$ 1.500,00

R\$ 115,00 em 10x

[Ver mais](#)

LG



R\$ 5.000,00

R\$ 500,00 em 10x

[Ver mais](#)

```
.products {  
  flex-direction: row;  
}
```

CSS: flexbox - propriedades

- ❖ Se usarmos `flex: 1 1 30%`, dizemos que a largura desse elemento é 30% do container pai (***flex-basis***: 30%), pode aumentar sua largura se estiver sobrando espaço (***flex-grow***: 1) e pode encolher se faltar espaço (***flex-shrink***: 1).

```
/* flex-items */  
.product-card {  
  flex: 1 1 30%;  
  /* ocupa ~30% da tela */  
  border: 1px solid #ccc;  
  padding: 10px;  
  text-align: center;  
}
```

TVs

As melhores TVs você encontra aqui

Samsung



R\$ 2.000,00

R\$ 200,00 em 10x

[Ver mais](#)

CSS: flexbox - propriedades

- ❖ Então usar `flex: 1 1 30%` é igual usar:
- ❖ *`flex-basis: 30%`, `flex-grow: 1` e `flex-shrink: 1`*

```
/* flex-items */  
.product-card {  
  flex: 1 1 30%;  
  /* ocupa ~30% da tela */  
  border: 1px solid #ccc;  
  padding: 10px;  
  text-align: center;  
}
```

```
/* flex-items */  
.product-card {  
  flex-basis: 30%;  
  flex-grow: 1;  
  flex-shrink: 1;  
  border: 1px solid #ccc;  
  padding: 10px;  
  text-align: center;  
}
```

CSS: flexbox - propriedades

- ❖ Se usarmos o *width* em vez do *flex-basis*, definimos uma largura base para o elemento, mais que pode ser ignorada caso tenha uma definição de *flex-basis* ou a necessidade de ajustar o espaço.
- ❖ Por exemplo, um container `display: flex`, tem três itens com *flex* 1 1 30% e se adaptam ao espaço disponível e quebram linha com *flex-wrap*.

```
/* flex-items */  
.product-card {  
  flex-basis: 30%;  
  flex-grow: 1;  
  flex-shrink: 1;  
  border: 1px solid #ccc;  
  padding: 10px;  
  text-align: center;  
}
```

TVs

Samsung



Preço: R\$ 2.000,00

Parcelamento: R\$
200,00 em 10x

[Ver mais sobre TVs
Samsung](#)

CSS: flexbox - propriedades

- ❖ Enquanto com width 30%, ele tentam manter esse valor mais rigidamente e podem gerar overflow ou comportamento menos previsível.

```
/* flex-items */
.product-card {
  width: 30%;
  /* ocupa ~30% da tela */
  border: 1px solid #ccc;
  padding: 10px;
  text-align: center;
}
```



CSS: flexbox - propriedades

- ❖ Como escolhemos o *flex-direction column*, aqui temos um filho por coluna.
- ❖ E o *gap* é o espaçamentos entre os itens filhos do parent flex.

```
display: flex;  
gap: 10px;  
flex-direction: column;
```

RS 2.000,00
RS 200,00 em 10x

[Ver mais](#)

LG



RS 5.000,00
RS 500,00 em 10x
[Ver mais](#)

Philco



```
display: flex;  
gap: 10px;  
flex-direction: row;
```

Samsung



RS 2.000,00
RS 200,00 em 10x
[Ver mais](#)

Philco



RS 5.000,00
RS 500,00 em 10x
[Ver mais](#)

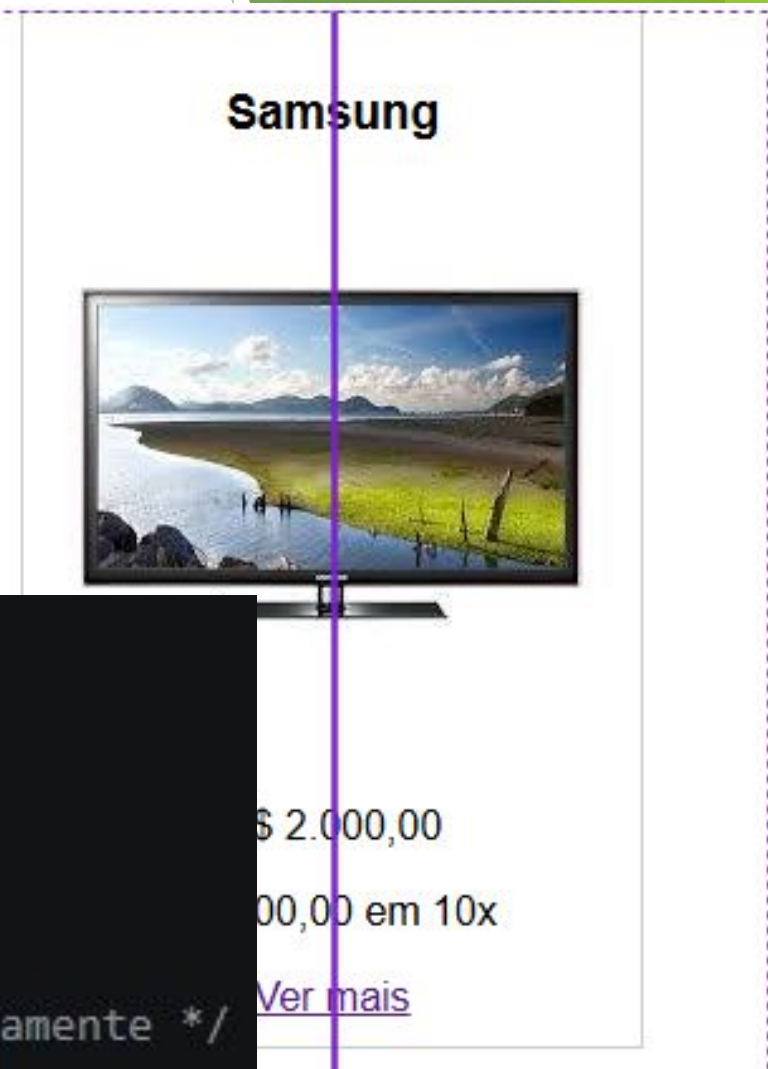


RS 1.500,00
RS 115,00 em 10x
[Ver mais](#)

CSS: flexbox - principais propriedades

- ❖ **Flex-wrap:** Determina se os itens devem quebrar para múltiplas linhas ou colunas.
- ❖ **Justify-content:** Alinha os itens ao longo do eixo principal.
- ❖ **align-items:** Alinha os itens ao longo do eixo cruzado (cross axis).

```
/* flex-container */  
.products {  
  display: flex;  
  gap: 20px;  
  flex-direction: column;  
  flex-wrap: wrap;  
  /* quebra linha automaticamente */  
  padding: 20px;  
  align-items: center;  
  justify-content: center;  
}
```



Fonte: autora, 2026

CSS: flexbox - propriedades

- ❖ O *justify-content center* centraliza todo o conteúdo dentro do pai flex de acordo com a dimensão do pai.
- ❖ E o *align-items center* centraliza todos os conteúdos dentro dos filhos do pai flex, mantendo as dimensões exclusivas de cada filho.

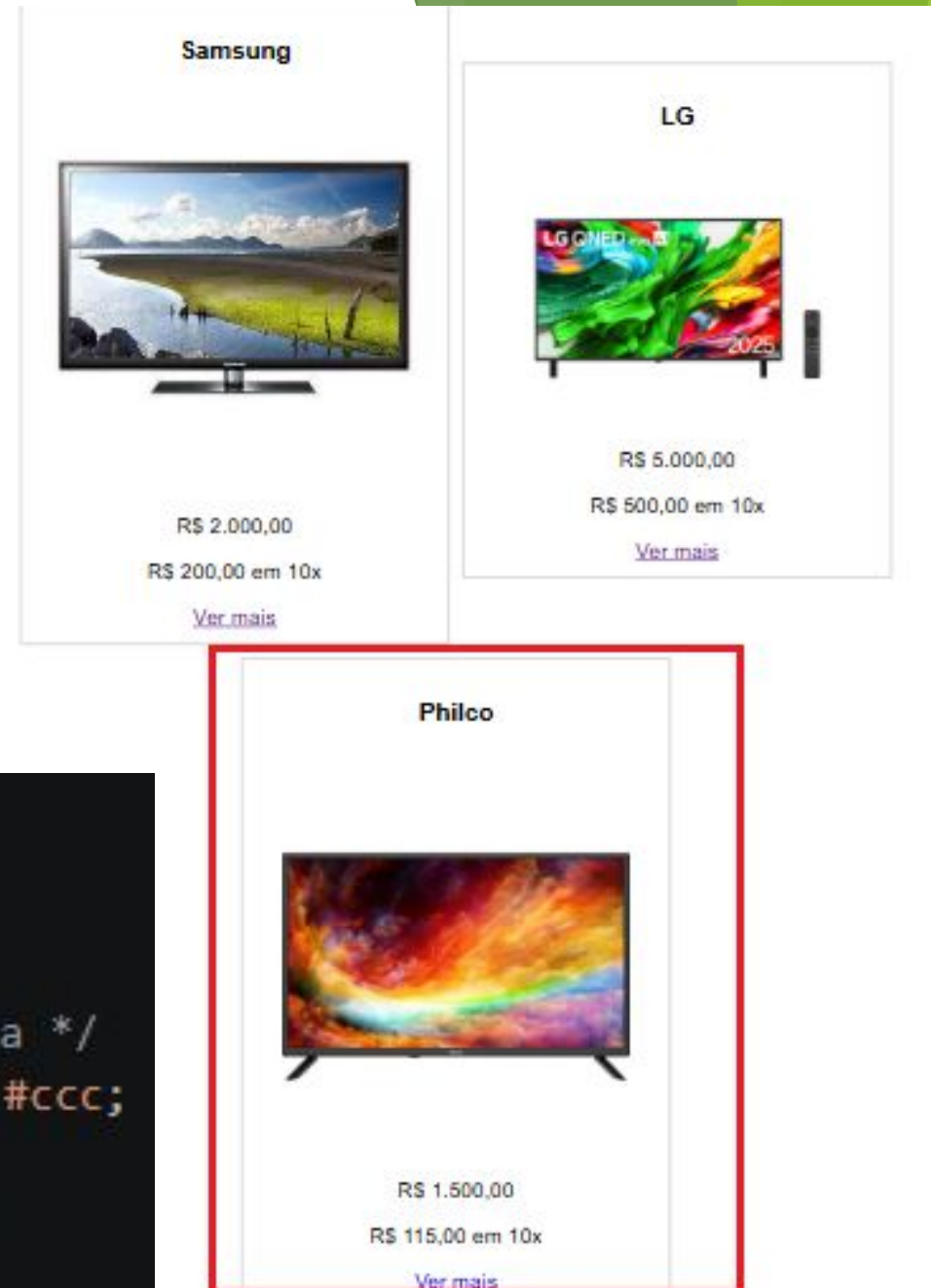


CSS: flexbox - propriedades

- ❖ E se usamos *flex-direction row* com *flex-wrap wrap*, teremos o efeito de um elemento cair para outra linha quando não tiver espaço de largura para ele ser exibido.

```
/* flex-container */  
.products {  
  display: flex;  
  gap: 10px;  
  flex-direction: row;  
  flex-wrap: wrap;  
  /* quebra linha automaticamente */  
  padding: 20px;  
  align-items: center;  
  justify-content: center;  
}
```

```
/* flex-items */  
.product-card {  
  flex: 1 1 30%;  
  /* ocupa ~30% da tela */  
  border: 1px solid #ccc;  
  padding: 10px;  
  text-align: center;  
}
```



CSS: flexbox - propriedades

- ❖ Também podemos alinhar os filhos individualmente com a propriedade *align-self*.

```
.products {  
  /* quebra linha automaticamente */  
  padding: 20px;  
  align-items: center;  
  justify-content: center;  
}  
  
.product-card.align {  
  align-self: stretch;  
}
```

Categoria de produtos

TVs

As melhores TVs você encontra aqui



The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Exercício

CSS: exercício 4

- ❖ all 1: no cabeçalho de todas as páginas, adicione o links de navegação (*nav*) dentro do cabeçalho (*header*).

```
<body class="page-category-tvs">
  <header class="header">
    <p>Venda de TVs e acessórios online</p>
    <nav class="nav">
      <ul>
        <li>
          <a href="./products.html">Home</a>
        </li>
        <li>
          <a href="./category_tvs.html" target="_self">TVs</a>
        </li>
        <li>
          <a href="./category_acessorios.html">Acessórios</a>
        </li>
      </ul>
    </nav>
  </header>
  > images
  <> category_acessorios.html
  <> category_tvs.html
  <> products.html
```

CSS: exercício 4

- ❖ all 2: estilize o cabeçalho principal da página para acompanhar a rolagem da página, apresentar a listagem de lado a lado e ter uma cor de fundo.

```
> images
<> category_acessories.html
<> category_tvs.html
<> products.html
# styles.css
```

```
/* all 2 */
.header {
  position: sticky;
  top: 0;
  z-index: 3;
  padding: 10px;
  background-color: #darkkhaki;
}

.header .nav {
  text-align: center;
}

.header .nav li {
  display: inline-block;
  margin-right: 10px;
}
```

CSS: exercício 4

- ❖ all 3: estilize o *body* da página para não ter espaçamentos, zerando a margin.
- ❖ all 4: adicione **espaçamento interno** de 20px para a tag main.

```
> images  
<> category_acessorios.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

```
/* all 3 */  
body {  
  margin: 0;  
}  
  
/* all 4 */  
main {  
  padding: 20px;  
}
```

CSS: exercício 4

- ❖ ct 1: substitua a tabela de produtos de tvs para usar tags semânticas (header, ul > li, article > header e footer, footer).

```
> images
<> category_acessorios.html
<> category_tvs.html
<> products.html
```

```
<section class="products" aria-labelledby="products-title">
  <header>
    <h2 id="products-title">TVs disponíveis</h2>
    <p>Confira preços e condições de parcelamento</p>
  </header>
  <ul class="products-list">
    <li>
      <article class="product">
        <header>...
        </header>

        <figure>...
        </figure>

        <p class="price">
          <strong>Preço:</strong>
          <data value="2000">R$ 2.000,00</data>
        </p>

        <p class="installment">
          <strong>Parcelamento:</strong>
          <data value="200">R$ 200,00</data> em 10x
        </p>

        <footer>...
        </footer>
      </article>
    </li>
```

CSS: exercício 4

- ❖ ct 2: para listagem de produtos, adicione a classe ***products-list*** e para cada produto, adicione a classe ***product***.
- ❖ ct 3: na listagem de produtos, adicione o atributo ***aria-labelledby*** com o valor do id “products-title”

```
> images
<> category_acessories.html
<> category_tvs.html
<> products.html
```

```
<section class="products" aria-labelledby="products-title">
  <header>
    <h2 id="products-title">TVs disponíveis</h2>
    <p>Confira preços e condições de parcelamento</p>
  </header>

  <ul class="products-list">
    <li>
      <article class="product">
        <header>...
        </header>

        <figure>...
        </figure>

        <p class="price">
          <strong>Preço:</strong>
          <data value="2000">R$ 2.000,00</data>
        </p>

        <p class="installment">
          <strong>Parcelamento:</strong>
          <data value="200">R$ 200,00</data> em 10x
        </p>

        <footer>...
        </footer>
      </article>
    </li>
```

CSS: exercício 4

- ❖ ct 4: Defina as propriedades do flex-container para classe *products-list*.
- ❖ ct 5: Defina as propriedades dos flex-items para classe *product*.

```
> images
<> category_acessorios.html
<> category_tvs.html
<> products.html
# styles.css
```

```
/* ct4 flex-container */
.products-list,
.products-list li {
  display: flex;
}

.products-list {
  gap: 10px;
  flex-direction: column;
  flex-wrap: nowrap;
  padding: 20px;
  align-items: stretch;
  justify-content: center;
}

/* ct5 flex-items */
.product {
  flex-basis: 30%;
  flex-grow: 1;
  flex-shrink: 1;
  border: 1px solid #ccc;
  padding: 10px;
  text-align: center;
}
```

CSS: exercício 4

- ❖ ct 6: utilize medidas responsivas nas imagens de produtos (max-height, max-width).
- ❖ ct 7: altere a orientação dos produtos de coluna para linhas para telas maiores que tablet (768px +) e defina para os produtos quebrarem linha caso neste caso (media, flex-direction, flex-wrap).

```
/* ct 6 */  
.product img {  
  max-height: 150px;  
  max-width: 100%;  
}  
  
/* ct 7 */  
@media (min-width: 768px) {  
  .products-list {  
    flex-direction: row;  
    flex-wrap: wrap;  
  }  
}
```

```
> images  
<> category_acessorios.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

CSS: exercício 4

❖ category_tvs.html



Salve esse código no seu email,
Google Drive ou GitHub!

Vídeo



Referência da aula

- ❖ DE MOOR, Thomas. CSS Grid vs Flexbox. X-Team, 10 set. 2019. Disponível em: <https://x-team.com/magazine/css-grid-vs-flexbox>. Acesso em: 30 mar. 2026.
- ❖ AGRAWAL, Ritika. Exploring CSS Flexbox: getting started with the basics. Medium, 2023. Disponível em: <https://medium.com/@RitikaAgrawal08/exploring-css-flexbox-getting-started-with-the-basics-1174eea3ad4e>. Acesso em: 30 mar. 2026.
- ❖ COYIER, Chris. A complete guide to Flexbox. CSS-Tricks, 14 fev. 2019. Disponível em: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>. Acesso em: 30 mar. 2026.

Referência da aula

- ❖ AGRAWAL, Ritika. Exploring CSS Flexbox: getting started with the basics. Medium, 2023. Disponível em: <https://medium.com/@RitikaAgrawal08/exploring-css-flexbox-getting-started-with-the-basics-1174eea3ad4e>. Acesso em: 30 mar. 2026.
- ❖ MAKE COMPUTER SCIENCE GREAT AGAIN. Understanding Flexbox: a comprehensive guide. Medium, 2023. Disponível em: <https://medium.com/@MakeComputerScienceGreatAgain/understanding-flexbox-a-comprehensive-guide-992bcd5f04de>. Acesso em: 30 mar. 2026.
- ❖ W3SCHOOLS. CSS Flexbox. W3Schools, [s.d.]. Disponível em: https://www.w3schools.com/css/css3_flexbox.asp. Acesso em: 30 mar. 2026.



univille.br